# Aula 01 - Prof. Vinícius Borges e Raphael Lacerda

*SEFAZ-AL (Auditor Fiscal da Administração Tributária Estadual - AFTE) Desenvolvimento de Sistemas - 2026 (Pós-Edital)*

Autor:
**Vinicius Borges**

27 de Agosto de 2026

02648216103 - HENRIQUE AMORIM BARRETO

# Índice

1) Java - Teoria ..... 3

2) Java - Questões Comentadas ..... 55

3) Java - Lista de Questões ..... 84



02648216103 - HENRIQUE AMORIM BARRETO

# Java

## Conceitos Gerais



Java é uma **linguagem de programação orientada a objetos**, criada pela empresa Sun Microsystems no início dos anos 1990, sendo oficialmente lançada em 1995. Seu principal objetivo era oferecer uma **linguagem que pudesse ser executada em diversos tipos de dispositivos e sistemas operacionais**, sem a necessidade de modificações no código-fonte. Esse princípio ficou conhecido como **"Write Once, Run Anywhere"** (WORA).

Essa propriedade foi habilitada graças à Java Virtual Machine (JVM), que falaremos mais à frente. Em 2010, a Sun Microsystems foi adquirida pela Oracle Corporation, que passou a ser a mantenedora da plataforma Java, que atualmente conta com mais de 3 bilhões de dispositivos rodando seus códigos.

Seus usos englobam:

- Aplicações para desktops e dispositivos móveis
- Aplicações web
- Servidores
- Jogos
- Entre outros

Porém, não podemos encarar o Java como apenas uma linguagem – ele é uma **plataforma completa de desenvolvimento**. Essa plataforma é composta por três partes principais: a **Java Virtual Machine**, o **Java Runtime Environment** e o **Java Development Kit**.

### Componentes da Plataforma Java

| Java Virtual Machine (JVM) | Java Runtime Environemnt (JRE) | Java Development Kit (JDK) |
|---|---|---|

A **JVM** é responsável por executar o bytecode gerado pelo compilador. Cada sistema operacional (Windows, Linux, macOS, etc.) possui sua própria implementação da JVM, o que garante a portabilidade dos programas.

O **JRE** contém a JVM, bibliotecas padrão da linguagem e outros recursos necessários para rodar aplicações. Não inclui ferramentas de desenvolvimento. Por fim, o kit de desenvolvimento Java (**JDK**) é um conjunto completo que inclui tudo o que está na JRE mais compilador, depurador e ferramentas para desenvolvimento de software.

A sintaxe do Java não é das "mais bonitas", e pode parecer um pouco complexa numa primeira interação. Veja um exemplo:

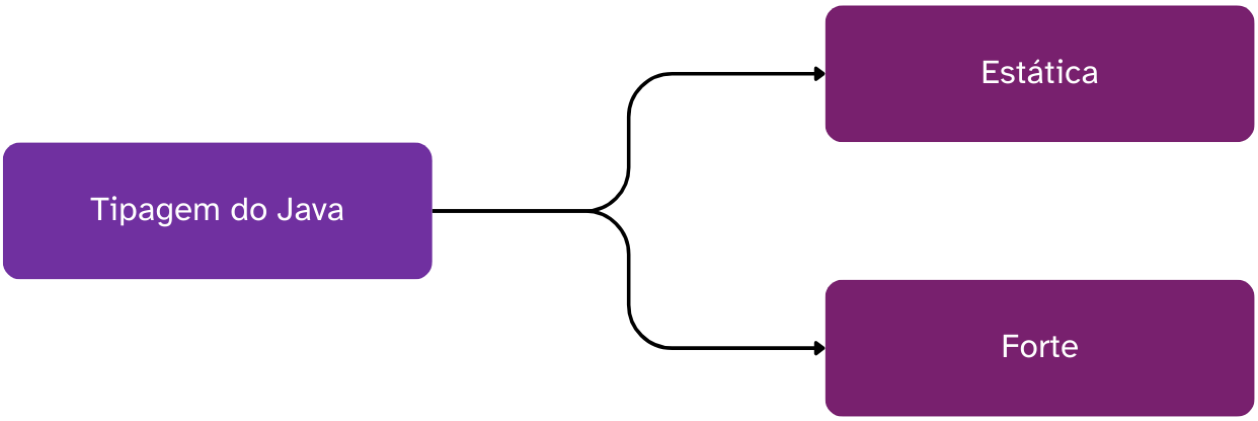
```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Olá, mundo!");
    }
}
```

Aqui:

- `public class OlaMundo`: define uma classe pública com o nome `OlaMundo`. Todo código Java deve estar dentro de uma classe.
- `public static void main(String[] args)`: este é o método principal. Ele é o ponto de entrada da aplicação.
- `System.out.println(...)`: imprime uma mensagem no console.

Todo esse código fica armazenado num arquivo .java, compondo o nosso programa básico.

Conforme o código apresentado, você deve ter percebido que o Java possui uma **tipagem estática**, isso é, os tipos das variáveis são verificados em tempo de compilação, e devem ser declarados explicitamente. No momento da compilação, o compilador Java verifica se os tipos são compatíveis e lança erros caso contrário. Isso reduz a incidência de erros de tipo em tempo de execução. Além disso, o Java possui **tipagem forte**, não permitindo conversões implícitas entre tipos incompatíveis sem perda de informação ou risco de erro.




Para essa aula, usaremos o **Java 21** (de 2023), por ser a última versão LTS (*Long-Term Support*).

*É ideal para essa aula que você possua conhecimentos de lógica de programação e sobre o paradigma orientado a objetos – informações básicas a esses tópicos serão tomadas como já aprendidas e não serão explicadas em profundidade.*

# Organização de Arquivos

O Java trabalha com um sistema completa de organização de arquivos, baseado em dois arquivos principais:

- `.java` → arquivo de código-fonte
- `.class` → arquivo compilado em bytecode, gerado pelo compilador Java (`javac`)

Então, em geral, o fluxo é o seguinte:

1. Escrevemos o código no arquivo `.java`.
2. Compilamos com a instrução de linha de comando `javac NomeDoArquivo.java`, gerando um arquivo .class.
3. Executamos o `.class` com o comando `java NomeDaClasse`.

Essas classes podem ser alocadas em um domínio em comum, denominado **pacote** – ou `packages`. Eles são basicamente conjuntos de classes organizadas em diretórios. Declaramos um pacote com, por exemplo, `package com.meuprojeto.modelo`, no início de cada arquivo .java – assim, criamos um elo entre os diferentes códigos. Uma organização padrão para arquivos Java em diretórios segue abaixo:

sh

```
meu-projeto/
├── src/                    # Código-fonte do projeto
│   ├── com/
│   │   ├── empresa/
│   │   │   ├── modelo/
│   │   │   │   ├── Cliente.java
│   │   │   │   ├── Produto.java
│   │   │   ├── servicos/
│   │   │   │   ├── PedidoService.java
│   │   │   │   ├── EstoqueService.java
│   │   ├── util/
│   │   │   ├── StringUtils.java
├── bin/                    # Arquivos compilados (.class)
├── lib/                    # Dependências externas (JARs)
├── docs/                   # Documentação do projeto
├── README.md               # Descrição do projeto
```

📂 `src/` → Contém todo o código-fonte (.java).
📂 `bin/` → Contém os arquivos .class gerados após a compilação.
📂 `lib/` → Contém bibliotecas externas (arquivos .jar).
📂 `docs/` → Contém a documentação gerada pelo javadoc.
📜 `README.md` → Explica o projeto.

# Sintaxe Java

Cada linha de código Java deve estar **dentro de uma classe**. Por padrão, essa classe **deve começar com uma letra maiúscula** – então, vamos tomar como exemplo o nome mais comum de classe: `Main`. O nome do arquivo Java **deve ser igual ao nome da classe**, só assim será possível executar o programa.

Então, obrigatoriamente, o código abaixo deve estar estruturado dentro de um arquivo denominado `Main.java`.

Java

```java
public class Main {
    public static void main(String[] args) {
        ...
    }
}
```

Outra obrigatoriedade presente nesse código é o **método `main(...)`**. Qualquer código dentro desse método será executado, obrigatoriamente.

Para fins de visualização de respostas, como temos o imprimir() na lógica de programação, print() em Python ou console.log() em JavaScript, no Java também temos um método específico: o `System.out.println()`. Ele irá exibir o parâmetro recebido dentro do terminal de comando. Podemos escrever "Olá, mundo!" da seguinte forma:

Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Olá, mundo!");
    }
}
```

```
Olá, mundo!
```

> OBS: um comando *print*, além do *println*, também existe. A diferença é que ele não insere uma nova linha ao fim do *output*.

Quanto às palavras chaves, como `public`, `static`, `void`, `String` e `args`, não se preocupe ainda, veremos todas mais à frente.

Além disso, o Java suporta **comentários**, tanto de **linha única**, que são caracterizados pela presença de barras duplas `//`, quanto de **múltiplas linhas**, enquadrados entre os símbolos `/*` e `*/`.



02648216103 - HENRIQUE AMORIM BARRETO

# Tipos de Dados e Variáveis

Como vimos, Java é uma linguagem fortemente tipada, e deve ter seus tipos declarados expressamente, determinando o espaço reservado na memória, o tipo de operações e a forma como eles serão tratados pelo compilador e pela máquina virtual. Os tipos do Java se dividem em **tipos primitivos** e **tipos referenciados**.

O Java possui 8 tipos primitivos. Esses tipos são nativos da linguagem, ou seja, não são objetos. Eles são armazenados diretamente na memória e representam valores simples.

| Tipos Numéricos Inteiros | | | |
|---|---|---|---|
| Tipo | Tamanho | Intervalo | Exemplo |
| byte | 8 bits | -128 a 127 | byte b = 10; |
| short | 16 bits | -32.768 a 32.767 | short s = 3000; |
| int | 32 bits | $-2^{31}$ a $2^{31}-1$ | int i = 123456; |
| long | 64 bits | $-2^{63}$ a $2^{63}-1$ | long l = 12345678900L; |

** O sufixo L é obrigatório para indicar literais long.*

| Tipos Numéricos de Ponto Flutuante | | | |
|---|---|---|---|
| Tipo | Tamanho | Intervalo | Exemplo |
| float | 32 bits | 6 a 7 dígitos | float f = 5.5f; |
| double | 64 bits | 15 a 16 dígitos | double d = 3.14159; |

** O sufixo f é obrigatório para float, pois números decimais são interpretados como double por padrão.*

| Tipo de Caractere | | | |
|---|---|---|---|
| Tipo | Tamanho | Intervalo | Exemplo |
| char | 16 bits | Representa um caractere Unicode | char letra = 'A'; |

| Tipo Booleano | |
|---|---|
| Tipo | Valores possíveis |
| boolean | `true` ou `false` |

Diferente dos tipos primitivos, os tipos por referência representam objetos. Isso significa que a variável armazena o endereço de memória onde o objeto está localizado, e não o valor diretamente. Exemplos comuns incluem:

- Strings (`String nome = "João";`)
- Arrays (`int[] numeros = {1, 2, 3};`)
- Classes personalizadas (`Pessoa p = new Pessoa();`)

- Classes wrapper (empacotadoras) como Integer, Double, Boolean, etc.]

Veja uma breve comparação entre os tipos:

| Característica | Tipo Primitivo | Tipo Referenciado |
|---|---|---|
| Armazenamento | Valor direto | Referência (endereço de memória) |
| Herança de Object | Não | Sim |
| Métodos disponíveis | Não | Sim (ex: toUpperCase() em String) |
| Comparação (==) | Compara valores diretamente | Compara referências (endereços) |

Um detalhe, para cada tipo primitivo, há uma **classe empacotadora** (ou *wrapper*) correspondente no pacote `java.lang`, que permite tratar os primitivos como objetos. Por exemplo, para o tipo primitivo `int`, tempos o wrapper `Integer`, para `char`, temos `Character`, para `boolean`, temos `Boolean`, dentre outras.

Isso acontece pois o Java foi projetado para balancear eficiência e orientação a objetos. Tipos primitivos são mais leves e performáticos, ideais para cálculos e uso intensivo de memória. Já os wrappers permitem que esses tipos também possam ser manipulados como objetos em contextos específicos.

Quanto aos nomes das variáveis, ou **identificadores**, temos algumas restrições:

- Nomes podem conter letras, dígitos, sublinhado e cifrão
- Nomes **devem** começar com uma letra, cifrão ou sublinhado
- Nomes, por convenção, usam camelCase, ou seja, iniciam-se com letra minúscula e usam letra maiúscula para iniciar cada palavra subsequente

Agora, podemos criar variáveis e atribuir valores a elas. A estrutura padrão é:

```
<tipo> <nome_da_variável> [= valor_inicial]
```

Vamos fazer alguns exemplos:

Java

```java
int idade; //variável declarada, mas não atribuido valor (inicializada)
double salarioLiquido = 10.99; //exemplo de camelCase
String nome = "Maria";
int x = 1, y = 2, z = 3; //múltiplas variáveis de mesmo tipo
```

Também é possível termos **delimitações de escopo**, que determina **onde uma variável é visível e pode ser utilizada**. Em Java, o escopo está diretamente ligado aos blocos de código delimitados por chaves {}. Podemos classificar os escopos em três categorias:

- **Escopo local:** uma variável local é declarada dentro de um método, construtor ou bloco (if, for, while, etc.) e só é acessível dentro desse bloco.

```java
public class Teste {
    public void exemplo() {
        int a = 10; // variável local
        if (a > 5) {
            int b = 20; // variável local dentro do if
            System.out.println(a + b); // válido
        }
        System.out.println(b); // ERRO! b não está visível aqui
    }
}
```

- **Escopo de Instância:** são variáveis declaradas dentro de uma classe, mas fora de qualquer método. Cada objeto criado a partir da classe possui sua própria cópia da variável.

```java
public class Pessoa {
    String nome;  // variável de instância
    int idade;    // variável de instância

    public void exibir() {
        System.out.println("Nome: " + nome);
    }
}
```

OBS: Variáveis de instância possuem valores padrão, caso não sejam inicializadas: números → `0` ou `0.0`, `boolean` → `false`, `char` → `\u0000`, `objetos` → `null`.

- **Escopo de Classe:** são declaradas com o modificador `static`, dentro da classe. Pertencem à classe, e não aos objetos. Todos os objetos compartilham a mesma variável.

```java
public class Contador {
    static int total = 0; // variável de classe (compartilhada)

    public Contador() {
        total++;
    }
}
```

Ainda, é possível fazer a declaração de **variáveis finais**. Isso impede que o valor da variável seja alterado após sua atribuição, criando uma espécie de constante em Java. A atribuição é feita a partir do modificador `final`. Veja:

```java
final double PI = 3.14159;
PI = 3.14; // ERRO: não é permitido alterar
```

**(QUADRIX/CFO/2025)** As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

Java não suporta tipos de dados primitivos, sendo obrigado a utilizar apenas objetos para representar valores como inteiros, caracteres e booleano

**Comentários:**

Pelo contrário... como vimos, temos tanto tipos primitivos quanto tipos referenciados (ou *wrappers*). *(Gabarito: Errado)*

## Generics

Generics (ou "tipos genéricos") são um recurso introduzido no Java 5 que permite especificar tipos de forma parametrizada. Com eles, podemos definir classes, interfaces e métodos que trabalham com tipos de forma flexível e segura, sem perder a verificação de tipo em tempo de compilação.

Antes do uso de Generics, muitas APIs usavam tipos genéricos como Object, o que exigia casting manual e podia gerar erros em tempo de execução. Com Generics, o compilador passa a verificar os tipos em tempo de compilação, proporcionando maior segurança e legibilidade.
Ou seja, ao invés de criar um código com algum tipo específico, podemos criar nossas próprias classes genéricas. Para isso, usamos a notação `<...>`, onde `...` é substituído pelo nome do tipo que queremos atribuir. Veja um exemplo:

Java

```java
public class Caixa<T> {
    private T conteudo;

    public void guardar(T item) {
        this.conteudo = item;
    }
    public T abrir() {
        return conteudo;
    }
}
```

Neste exemplo, a classe Caixa pode aceitar qualquer tipo de objeto <T>. O tipo <T> é um parâmetro genérico.

E sua respectiva aplicação:

Java

```java
Caixa<String> caixa1 = new Caixa<>();
caixa1.guardar("Livro");
String item = caixa1.abrir();

Caixa<Integer> caixa2 = new Caixa<>();
caixa2.guardar(42);
Integer numero = caixa2.abrir();
```

Assim, criamos uma instância da classe Caixa, usando o tipo String, que substitui o tipo genérico que previmos originalmente.

# Operadores

Operadores são empregados para aplicar operações diversas dentro da linguagem. Temos operadores aritméticos, de atribuição, de comparação, lógicos e *bitwise*. Veja:



## ESQUEMATIZANDO

| Operadores Aritméticos | | | |
|---|---|---|---|
| Operador | Nome | Descrição | Exemplo |
| + | Adição | Soma dois valores | x + y |
| - | Subtração | Subtrai um valor de outro | x – y |
| * | Multiplicação | Multiplica dois valores | x * y |
| / | Divisão | Divide um valor por outro | x / y |
| % | Módulo | Retorna o resto da divisão | x % y |
| ++ | Incremento | Incrementa o valor da variável em 1 | ++x ou x++ |
| -- | Decréscimo | Diminui o valor da variável em 1 | --x ou x-- |

| Operadores de Atribuição | | |
|---|---|---|
| Operador | Exemplo | Equivalência |
| = | x = 5 | x = 5 |
| += | x += 3 | x = x + 3 |
| -= | x -= 3 | x = x – 3 |
| *= | x *= 3 | x = x * 3 |
| /= | x /= 3 | x = x / 3 |
| %= | x %= 3 | x = x % 3 |

| Operadores de Comparações | | |
|---|---|---|
| Operador | Nome | Exemplo |
| == | Igual a | 5 == 5 |
| != | Diferente de | 5 != 4 |
| > | Maior que | 5 > 4 |
| < | Menor que | 4 < 5 |
| >= | Maior ou igual a | 5 >= 5 |
| <= | Menor ou igual a | 5 <= 5 |

| Operadores Lógicos | | |
|---|---|---|
| Operador | Nome | Descrição |
| && | E lógico | Retorna true se ambos os valores forem true |
| \|\| | OU lógico | Retorna true se ao menos um valor for true |
| ! | NÃO lógico | Nega o valor lógico de um atributo |

As operações *bitwise* são pouco úteis e não costumam ser exploradas em Java.

Uma outra observação, em comparações entre objetos, como String ou Integer, o operador `==` irá realizar uma comparação entre referências, e não entre conteúdo. Para comparação de conteúdo, usa-se o método `.equals()`.

**(Inédita/Prof. Felipe Mathias)** O código abaixo, escrito em Java, exibirá em tela o valor true.

```
public class ComparacaoStrings {
   public static void main(String[] args) {
      String a = "Java";
      String b = "Java";

      System.out.println(a.equals(b));
   }
}
```

**Comentários:**

Perfeito! Estamos comparando duas strings iguais, e, portanto, o print retornará *true*. *(Gabarito: Certo)*

# Strings e Operações

Em Java, uma `String` representa uma sequência de caracteres Unicode. As strings são classes que pertencem ao pacote `java.lang` (de importação automática), e possuem algumas características únicas:

- **É imutável:** uma vez criada, seu conteúdo não pode ser alterado.
- **É final:** não pode ser estendida por outra classe.
- É amplamente otimizada para manipulação de textos.

A imutabilidade da String garante segurança, performance em cache (por exemplo, no pool de strings), e evita efeitos colaterais indesejados quando a mesma string é compartilhada entre diferentes partes do sistema.

Podemos criar as strings de duas formas – a forma literal, que é mais otimizada e leva as strings para serem armazenadas no String Pool, uma área especial de memória na JVM, permitindo reaproveitamento se outra string igual for criada, e com o construtor `new`, menos eficiente, mas ainda válido.

Java

```java
String nome = "Java";

String nome = new String("Java");
```

As strings carregam alguns métodos consigo. Vamos organizá-los numa tabela.

| Método | Descrição |
|---|---|
| `.length()` | Retorna o comprimento de uma string |
| `.charAt(int index)` | Retorna o caractere em uma posição específica |
| `.substring(int começo, int fim)` | Retorna uma parte da string |
| `.equals(String var)` | Compara o conteúdo textual, retornando um booleano |
| `.compareTo(String var)` | Compara lexicograficamente duas strings, retornando 0 se forem iguais, negativo se a primeira for “menor” e positivo se a primeira for “maior” |
| `.indexOf()` | Posição da ocorrência |
| `.lastIndexOf()` | Último índice da ocorrência |
| `.split(String regex)` | Quebra a string com base em expressão regular |
| `.concat(String var)` | Concatena duas strings |

Lembra que falamos que as strings são imutáveis, né? Então, métodos que parecem alterar as strings, como colocá-las em maiúsculas (`.toUpperCase()`), na verdade estão criando novos objetos.

Acontece que muitas alterações podem levar a uma queda de desempenho do programa, já que novos espaços de memória estão sendo ocupados. Se o programa exige muitas alterações dinâmicas em texto, o uso de `StringBuilder` (não sincronizado) ou `StringBuffer` (sincronizado) é recomendado.

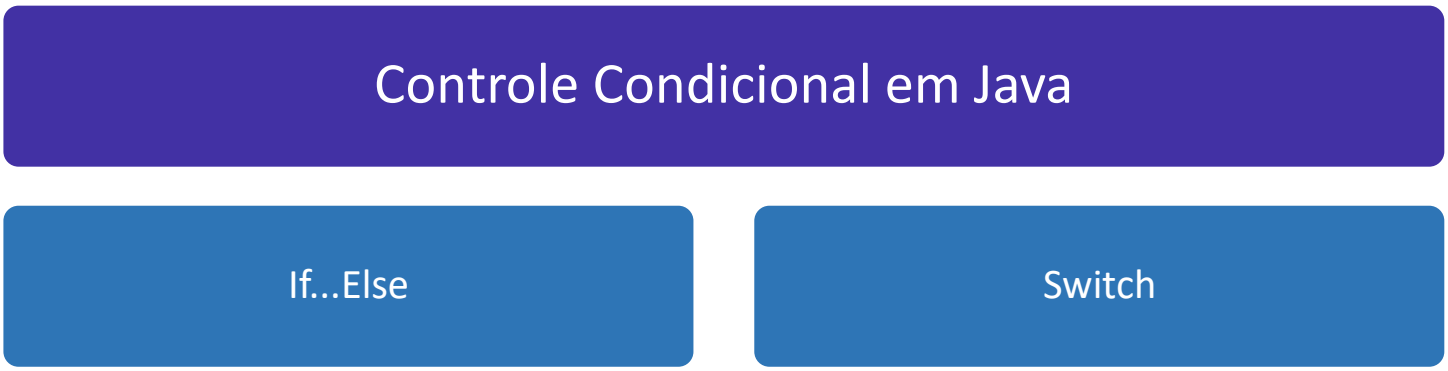
Java

```java
StringBuilder sb = new StringBuilder();
sb.append("Olá");
sb.append(" ");
sb.append("mundo!");
System.out.println(sb.toString()); // "Olá mundo!"
```

A classe `StringBuilder` é mutável e mais eficiente em concatenações repetidas.

# Controle Condicional

O controle condicional modifica o fluxo de execução do código, permitindo que um ou mais blocos sejam escolhidos para execução, dependendo do atendimento a condições específicas. O Java fornece duas estruturas de controle condicional: `If...else` e `Switch`.




## If...Else

A estrutura condicional if-else é usada quando precisamos executar um bloco de código apenas se uma condição for satisfeita. Ela pode ser complementada com múltiplas cláusulas else if, formando uma cadeia de decisões.

A sintaxe base da condicional é:

```
1    if (condição) {
2        // bloco executado se a condição for verdadeira
3    } else {
4        // bloco executado se a condição for falsa
5    }
```

Um exemplo prático:

Java

```java
int idade = 20;

if (idade >= 18) {
    System.out.println("Maior de idade.");
} else {
    System.out.println("Menor de idade.");
}
```

```
Maior de idade.
```

É possível estender o `if...else` para comportar mais verificações com o `else if`. Lembrando, as verificações são feitas uma a cada vez:

Java

```java
int nota = 75;

if (nota >= 90) {
    System.out.println("Conceito A");
} else if (nota >= 80) {
    System.out.println("Conceito B");
} else if (nota >= 70) {
    System.out.println("Conceito C");
} else {
    System.out.println("Reprovado");
}
```

```
Conceito C
```

O compilador segue o fluxo – interpretamos a primeira condição, caso seja falsa, seguimos em frente até encontrar um bloco verdadeiro. Se o encontrarmos, acabamos a execução ali; caso contrário, vamos até o bloco residual (`else`), e executamos o seu bloco de código.

## Switch

O `switch` é especialmente útil quando precisamos comparar o valor de uma variável contra múltiplos casos fixos. Em cenários assim, ele pode tornar o código mais limpo e organizado do que uma longa cadeia de `if-else if`. Veja a sintaxe base:

```
1    switch (expressao) {
2        case valor1:
3            // comandos
4            break;
5        case valor2:
6            // comandos
7            break;
8        default:
9            // comandos para qualquer outro valor
10   }
```

- `expressao`: pode ser char, byte, short, int, String, enum, ou wrapper types como Integer, Character, etc.
- `case`: valor literal a ser comparado com a expressão.
- `break`: usado para interromper o fluxo do switch. Sem ele, a execução cai para os próximos case (*fall through*).
- `default`: bloco executado se nenhum dos casos for correspondente. Opcional, mas recomendado.

A partir do Java 14, e estável no Jva 17, temos uma sintaxe mais concisa, utilizando `->` para construção, além de suporte a expressões switch com retorno de valor. Veja um exemplo:

Java

```java
int dia = 2;

 String resultado = switch (dia) {
    case 1 -> "Domingo";
    case 2 -> "Segunda-feira";
    case 3 -> "Terça-feira";
    case 4 -> "Quarta-feira";
    case 5 -> "Quinta-feira";
    case 6 -> "Sexta-feira";
    case 7 -> "Sábado";
    default -> "Desconhecido";
};
System.out.println("O dia correspondente é: " + resultado);
```

```
O dia correspondente é: Segunda-feira
```




**(CEBRASPE/CAGEPA/2024)**

```java
public class Cagepa {
    public static void main(String[] args) {
        Cagepa ip = new Cagepa();
        System.out.println("Resultado=" + ip.Cagepa(5));
    }

    private int Cagepa(int x) {
        if (x == 0)
            return 0;

        if (x % 2 != 0)
            return x + Cagepa(x - 1) + 4;

        return Cagepa(x - 1);
    }
}
```

Considerando o código precedente, desenvolvido em Java, assinale a opção que corresponde à correta execução desse código.

a) Resultado=12
b) Resultado=21
c) Resultado=4
d) Resultado=9
e) Resultado=14

**Comentários:**

Estamos rodando o método Cagepa(int x), que é um método recursivo – se x == 0, retorna 0, se o resto de x por 2 for diferente de 0, retornará a recursão, caso contrário, retorna Cagepa(x – 1). Estamos rodando a função com x = 5, então, como o resto da divisão de 5 por 2 é diferente de 0, caímos no segundo if:

Cagepa(5) → return (5 + 4) + Cagepa(4)
Cagepa(4) → return Cagepa(3) *(pois x % 2 = 0, caímos no residual)*
Cagepa(3) → return (3 + 4) + Cagepa (2)
Cagepa(2) → return Cagepa(1)
Cagepa(1) → return (1 + 4) + Cagepa (0)
Cagepa(0) = 0

Agora, podemos subir somando.

Cagepa(0) = 0
Cagepa(1) = 5
Cagepa(2) = Cagepa(1) = 5
Cagepa(3) = 7 + 5 = 12
Cagepa(4) = 12
Cagepa(5) = 9 + 2 = 21

Portanto, o output será “Resultado=21”. *(Gabarito: Letra B)*

# Estruturas de Repetição

Estruturas de repetição envolvem blocos que podem ser repetidos por um número definido ou indefinido de vezes, baseados em contador, condições ou outros. O Java apresenta duas estruturas de repetição: `While` e `For`.

## While

A estrutura `while` executa um bloco de código zero ou mais vezes, desde que a condição booleana avaliada antes de cada iteração seja true. Assim que a condição se tornar false, a repetição é interrompida.

```
    while (condição) {
        // bloco de código a ser repetido
    }
```

A condição deve ser obrigatoriamente uma expressão booleana (true ou false). Qualquer outro tipo causará erro de compilação. Veja um exemplo simples:

Java

```java
int contador = 1;

while (contador <= 5) {
    System.out.println("Contando: " + contador);
    contador++;
}
```

```
Contando: 1
Contando: 2
Contando: 3
Contando: 4
Contando: 5
```

Ainda, é possível a implementação do `do...while`. Diferentemente `do while` padrão, executaremos o bloco de código definido em `do {}` sempre uma vez, e a verificação é feita em seguida. Havendo verificação de um valor verdadeiro, seguimos nas repetições até que a condição se torne falsa – realizando a verificação sempre após a execução.

```
    do {
        // bloco de instruções
    } while (condição);
```

Veja um exemplo:

Java

```java
int contador = 1;

do {
    System.out.println("Contando: " + contador);
    contador++;
} while (contador <= 5);
```

```
Contando: 1
Contando: 2
Contando: 3
Contando: 4
Contando: 5
```

**(FGV/Pref. Niterói/2023)** Observe o seguinte programa elaborado na linguagem Java.

```
public class Main {
   public static void main(String[] args) {
     int i = 0;
     while (i < 3) {
        i++;
     }
      System.out.println(i *= 3);
   }
}
```

O resultado da execução do programa é:

a) 6
b) 7
c) 8
d) 9
e) 10

**Comentários:**

Questão tranquila. Estamos rodando um loop while que será executando enquanto i < 3, que parte de 0 e é incrementado em uma unidade a cada iteração (i++). Com isso, teremos 3 iteraçõs:

Entrada: i = 0; Ação: i++; Saída: i = 1
Entrada: i = 1; Ação: i++; Saída: i = 2
Entrada: i = 2: Ação: i++; Saída: i = 3

O print será i *= 3, ou i = i * 3, resultando em 3 * 3 = 9. *(Gabarito: Letra D)*

# For

O `for` é uma estrutura de repetição usada quando sabemos a quantidade de repetições que queremos. Ele é aplicável quando conhecemos o ponto de partida, a condição de parada e a o passo (que pode ser um incremento ou decremento).

A sintaxe base é:

```
for (inicialização; condição; atualização) {
    // bloco de código a ser repetido
}
```

Nessa estrutura:

- Inicialização: executada uma única vez no início.
- Condição: verificada antes de cada iteração. Se for false, o laço termina.
- Atualização: executada ao final de cada iteração.

Ao invés de um exemplo, vamos partir diretamente para uma questão e exploraremos esse tipo de estrutura.

**(CEBRASPE/SEPLAG CE/2024)** Considerando linguagens de programação, julgue o item que se segue.

O código Java a seguir, após executado, apresentará 140 como resultado.

```
public class Main{
  public static void main(String[] args) {
    int soma=0;
    for (int i = 5; i >= 1; i--) {
      for (int j = 1; j <= i; j++) {
        soma += j*i;
    }
  }
  System.out.println(soma);
 }
}
```

**Comentários:**

O código apresenta dois laços *for*, um externo e outro interno.

Para o laço externo, vamos de i = 5 até j = 1, com decréscimos unitários. Já para o laço interno, iremos de j = 1 até j = i, tendo uma quantidade variável de repetições. Por exemplo, na primeira iteração do laço externo, com i = 5, teremos j indo de 1 a 5.

Dessa forma, teremos as seguintes operações:

**LAÇO EXTERNO 1:** i = 5; j = 1 até j = 5;

i = 1 → soma = 0 + 5*1 = 5
i = 2 → soma = 5 + 5*2 = 15
i = 3 → soma = 15 + 5*3 = 30
i = 4 → soma = 30 + 5*4 = 50
i = 5 → soma = 50 + 5*5 = 75

**LAÇO EXTERNO 2:** i = 4; j = 1 até j = 4;
i = 1 → soma = 75 + 4*1 = 79
i = 2 → soma = 79 + 4*2 = 87
i = 3 → soma = 87 + 4*3 = 99
i = 4 → soma = 97 + 4*4 = 115

**LAÇO EXTERNO 3:** i = 3; j = 1 até j = 3;
i = 1 → soma = 115 + 3*1 = 118
i = 2 → soma = 118 + 3*2 = 124
i = 3 → soma = 124 + 3*3 = 133

**LAÇO EXTERNO 4:** i = 2; j = 1 até j = 2/
i = 1 → soma = 133 + 2*1 = 135
i = 2 → soma = 135 + 2*2 = 139

**LAÇO EXTERNO 5:** i = 1; j = 1;
i = 1 → soma = 139 + 1*1 = 140

Portanto o item está correto, a execução do código terá como retorno o valor 140. *(Gabarito: Certo)*

É possível também a aplicação de uma estrutura um pouco mais avançada do `for`, chamada de **`for each`**, ou `for` estendido. Esse tipo de laço é uma evolução sintática do laço for tradicional, introduzido no Java 5, que facilita a iteração sobre coleções (como listas, conjuntos, mapas) e arrays, de forma mais segura, mais limpa e mais legível — especialmente quando não precisamos acessar o índice dos elementos.

A estrutura base é:

```
for (Tipo elemento : estruturaDeDados) {
    // bloco de código a ser executado com o elemento atual
}
```

Onde:

- **Tipo:** tipo de dado dos elementos da estrutura (ex: int, String, Aluno, etc.)
- **elemento:** variável temporária que recebe cada valor da estrutura, um por vez.
- **estruturaDeDados:** pode ser um array ou uma coleção que implemente a interface Iterable (como List, Set, etc.).

Veja o seguinte exemplo:

Java

```java
int[] numeros = {10, 20, 30, 40};

for (int n : numeros) {
    System.out.println(n);
}
```

```
10
20
30
40
```

Em cada iteração, a variável `n` assume o valor de um elemento do array. O loop percorre o array do primeiro ao último elemento, sem a necessidade de usar um índice manual como `numeros[i]`.

**(CEBRASPE/TRF 6/2025)** Julgue o próximo item, relativo a Java.

```
import java.io.*;
class Easy
{
     public static void main(String[] args)
     {
       int x[] = {1, 3, 7, 22, 51};
       for (int size : x)
          System.out.print(size);
     }
}
```

O resultado da execução do código precedente, desenvolvido em Java, será 6.

**Comentários:**

A questão envolve o uso do `for each` em Java. Estamos chamando cada elemento da estrutura `x` de "`size`", e o printando – o resultado será 1372251 (cada valor da sintaxe concatenado, pois usamos `print` e não `println`) – portanto, item incorreto. *(Gabarito: Errado)*

# Estruturas de Dados

Estruturas de dados são formas organizadas e eficientes de armazenar, manipular e acessar conjuntos de informações em memória. Elas definem como os dados são organizados internamente e influenciam diretamente o desempenho de algoritmos, operações de busca, inserção, remoção, ordenação e iteração.

Em Java, as estruturas de dados podem ser divididas em duas grandes categorias: as estruturas básicas nativas da linguagem, como os arrays, e as estruturas mais avançadas da API Java Collections Framework, como List, Set, Map, Queue, entre outras. Vamos explorar essas estruturas agora.

## Array

As **arrays** são estruturas de dados **estáticas e homogêneas**, isso é, devem ter somente um tipo de dado e não podem ter seu tamanho alterado depois de declarados. Além disso, essa estrutura armazenada os dados em posições sequenciais numeradas, ou **índices**, com o primeiro elemento partindo da **posição 0**.

| | |
|---|---|
| **ARRAY** | Conjunto **estático** e **ordenado** |
| | Unidimensional |
| | Permite um único tipo |
| | Elementos **acessados pelo índice** |

A sua declaração e inicialização pode ser feita sem valores ou com valores diretamente. A estrutura genérica segue a seguinte estrutura:

```
    <tipo>[] <nome> = new <tipo>[tamanho]
```

Veja exemplos:

Java

```java
int[] numeros = new int[5]; // cria um array de 5 inteiros (valores padrão: 0)

int[] numeros = {10, 20, 30, 40, 50}; // cria um array e atribui os valores
```

O acesso a cada elemento é feito a partir do seu índice. Por exemplo, o índice `numeros[0]`, no exemplo acima, retornaria `10` – o primeiro elemento. Por essa razão, o acesso é direto, e a complexidade computacional para acesso é de O(1).

**(CEBRSAPE/MPO/2024)** A respeito de linguagens de programação, julgue o item seguinte.

A execução da instrução abaixo, escrita em Java, gerará um erro, pois a inicialização de um array bidimensional deve ter todos os elementos com as mesmas quantidades.

int[][] matriz = {{1, 2}, {3, 4, 5}}

**Comentários:**

Temos uma matriz de inteiros – sendo que uma matriz representa uma *array* de duas dimensões. Não temos o objeto "matriz" propriamente dito, mas sim uma "array de arrays", cada uma agindo de forma independente da outra, não havendo restrição quanto à quantidade de elementos em cada dimensão. *(Gabarito: Errado)*

# Listas

Em Java, uma **lista** é uma **estrutura de dados linear**, **ordenada e indexada**, que permite armazenar um **conjunto de elementos do mesmo tipo**, geralmente permitindo elementos duplicados e mantendo a ordem de inserção.

O comportamento fundamental das listas está definido na interface `java.util.List<E>`, que faz parte do Java Collections Framework (JCF). Essa interface fornece um conjunto padronizado de operações como:

- Inserção (`add`, `addAll`),
- Remoção (`remove`, `clear`),
- Acesso por índice (`get`, `set`),
- Busca de elementos (`contains`, `indexOf`, `lastIndexOf`),
- Iteração;

A partir dessa interface, o Java oferece diversas implementações concretas, cada uma com características internas específicas de estrutura e desempenho.

Todas as listas em Java compartilham as seguintes propriedades fundamentais (salvo exceções específicas):

- Ordem mantida (elementos seguem a ordem de inserção);
- Elementos duplicados permitidos;
- Elementos acessados por índice (`get(int)` e `set(int)` são comuns);
- Homogeneidade garantida por generics (`List<T>`);
- Crescimento dinâmico (com exceção de listas de tamanho fixo);
- Podem armazenar `null` (com exceções específicas como CopyOnWriteArrayList e listas imutáveis do `List.of()`).

Veja uma tabela que resume as principais implementações de listas:

| Tipo de Lista | Estrutura | Vantagens | Limitações | Uso |
|---|---|---|---|---|
| ArrayList | Array dinâmico | Acesso rápido por índice (O(1)), leve, eficiente na leitura | Inserções/remoções no meio são lentas (O(n)), não é thread-safe | Listas comuns com muita leitura e acesso aleatório |
| LinkedList | Lista duplamente encadeada | Inserções e remoções eficientes nas pontas e no meio (O(1) em ponteiros) | Acesso por índice é lento (O(n)), maior consumo de memória | Filas, pilhas, listas com inserções frequentes |
| Stack | Subclasse de Vector | Implementa estrutura de pilha (LIFO) com métodos push e pop | Limitado semanticamente à pilha; performance do Vector | Quando a lógica exige comportamento de pilha |
| CopyOnWrite ArrayList | Array com cópia por escrita | Seguro para leitura concorrente, leitura rápida | Cópia completa do array em cada modificação → custo alto | Leitura frequente em ambientes concorrentes |
| List.of(...) (imutable) | Lista imutável (Java 9+) | Leve, segura, não permite alterações (excelente para constantes) | Não permite null, inserções, remoções ou modificações | Listas constantes, configurações, coleções fixas |

Assim, a interface List<E> irá apresentar o contrato padrão de listas no Java. A escolha da implementação envolve o comportamento esperado da aplicação:

- Acesso aleatório rápido → ArrayList
- Inserção/remoção frequente → LinkedList
- Concorrência segura → CopyOnWriteArrayList
- Pilha tradicional → Stack
- Imutabilidade e segurança → List.of(...)

Veja um exemplo de código de uma LinkedList:

Java

```java
import java.util.LinkedList;

public class ExemploLinkedList {
    public static void main(String[] args) {
        LinkedList<String> nomes = new LinkedList<>();

        nomes.add("Ana");
        nomes.add("Bruno");
        nomes.add("Carlos");

        nomes.addFirst("Início");
        nomes.addLast("Final");

        System.out.println("Lista: " + nomes);
        //saída -> Lista: [Início, Ana, Bruno, Carlos, Final]
    }
}
```

## Mapas

Um **Map** (ou Mapa) é uma estrutura de dados que **armazena pares de chave e valor**, permitindo a associação entre um identificador único (a chave) e uma informação correspondente (o valor). Esses mapas compartilham algumas propriedades em comum:

- Cada chave (key) é única;
- Cada chave é associada a um único valor;
- Os valores podem se repetir;
- O acesso é feito pela chave, não por índice;
- Não implementa a interface `Collection`, mas é parte do `Java Collections Framework`;
- Define o contrato de comportamento na interface `java.util.Map<K, V>`.

Da mesma forma que as listas possuem uma gama de métodos associados, os Maps também trazem os seus.

| Método | Estrutura |
|---|---|
| put(K chave, V valor) | Associa a chave ao valor (substitui se já existir) |
| get(K chave) | Retorna o valor associado à chave |
| containsKey(K chave) | Verifica se a chave existe |
| containsValue(V valor) | Verifica se o valor existe |
| remove(K chave) | Remove o par chave-valor |
| keySet() | Retorna um Set com todas as chaves |
| values() | Retorna uma Collection com todos os valores |

| | |
|---|---|
| entrySet() | Retorna um Set com os pares chave-valor (Map.Entry) |
| size() | Retorna o número de pares armazenados |
| clear() | Remove todos os pares do mapa |

Temos também diferentes tipos de implementação de mapas.

| Tipo de Mapa | Estrutura | Ordenação | Permite null | Uso |
|---|---|---|---|---|
| HashMap | Tabela Hash (hash table) | Não mantém ordem | 1 chave null, vários valores null | Map padrão, rápido para chave/valor |
| LinkedHashMap | Hash + Lista duplamente ligada | Ordem de inserção | Sim | Quando ordem de inserção importa |
| TreeMap | Árvore binária balanceada (Red-Black) | Ordenado por chave (Comparable ou Comparator) | Não permite null como chave | Ordenação natural ou personalizada por chave |
| Hashtable | Tabela Hash (legado) | Não ordenado | Não permite null (nem chave nem valor) | Código legado com múltiplas threads |
| ConcurrentHashMap | Segmentos de Hash sincronizados | Não ordenado | Não permite null (nem chave nem valor) | Alto desempenho com segurança para múltiplas threads |
| EnumMap | Array interno com enum ordinal | Ordenado pela ordem do enum | Chave não pode ser null | Chaves baseadas em enum, extremamente leve e rápida |

Na prática, o formato mais utilizado é o HashMap. Veja um exemplo de construção:

Java

```java
import java.util.HashMap;

public class ExemploMap {
    public static void main(String[] args) {
        HashMap<Integer, String> usuarios = new HashMap<>();

        usuarios.put(1, "Ana");
        usuarios.put(2, "Carlos");
        usuarios.put(3, "Beatriz");

        System.out.println(usuarios.get(2)); // Carlos

        usuarios.remove(1); // Remove chave 1
        }
    }
}
```

## Sets

A interface `java.util.Set<E>` representa uma **coleção que não permite duplicatas**, ou seja, cada elemento inserido no Set deve ser único segundo um critério de igualdade. Suas propriedades são:

- Não permite elementos duplicados (com base no método `equals()` e `hashCode()`);
- Não há garantia de ordem, a menos que seja uma implementação que mantenha ordenação (como `TreeSet` ou `LinkedHashSet`);
- Pode armazenar `null` (dependendo da implementação);
- Pertence ao Java Collections Framework;
- Herda de Collection, portanto possui métodos como `add`, `remove`, `contains`, `isEmpty`, `size`, entre outros.

A interface Set é ideal quando o objetivo é garantir unicidade dos elementos, e não há necessidade de associar chaves a valores, como em Map.

```java
import java.util.HashSet;

public class ExemploSet {
    public static void main(String[] args) {
        HashSet<String> nomes = new HashSet<>();

        nomes.add("Ana");
        nomes.add("Carlos");
        nomes.add("Ana"); // duplicado, será ignorado

        System.out.println(nomes); // Ordem não garantida
    }
}
```

Também temos diferentes implementações concretas do Set. Veja:

| Tipo de Set | Estrutura | Permite Null | Ordenação | Uso |
|---|---|---|---|---|
| HashSet | Tabela hash (hash table) | Sim (1 valor) | Não ordenado | Uso geral, quando a ordem não importa |
| LinkedHashSet | Hash + lista duplamente ligada | Sim (1 valor) | Ordem de inserção | Quando a ordem de inserção é importante |
| TreeSet | Árvore binária balanceada (Red-Black Tree) | Não | Ordenado (natural ou Comparator) | Quando é necessário manter os elementos ordenados |
| EnumSet | Bit set especializado | Não | Ordem natural do enum | Quando os elementos são enum |
| CopyOnWriteArraySet | Array com cópia por escrita | Não | Ordem de inserção | Concorrência com leitura intensiva |

## Iterator

O **Iterator** é uma **interface genérica** presente no pacote `java.util` que fornece uma forma padronizada de **percorrer os elementos de uma coleção** de forma sequencial, sem expor sua estrutura interna. Ele é especialmente útil quando desejamos iterar sobre coleções como `List`, `Set`, `Queue` e remover elementos com segurança durante a iteração, evitando exceções como `ConcurrentModificationException`.

A interface `Iterator<E>` traz consigo alguns métodos:

- `boolean hasNext()` → Verifica se há mais elementos na coleção
- `E next()` → Retorna o próximo elemento
- `void remove()` → Remove o último elemento retornado por `next()`

Java

```java
import java.util.*;

public class ExemploIterator {
    public static void main(String[] args) {
        List<String> nomes = new ArrayList<>();
        nomes.add("Ana");
        nomes.add("Carlos");
        nomes.add("Beatriz");

        Iterator<String> it = nomes.iterator();

        while (it.hasNext()) {
            String nome = it.next();
            System.out.println(nome);
        }
    }
}
```

# Exceções

**Exceções** são **eventos que ocorrem durante a execução do programa**, **interrompendo o fluxo normal** das instruções quando ocorre uma situação anormal ou erro. Essas situações podem variar desde erros de lógica do programador (como acessar um índice inválido de um array) até falhas em recursos externos (como arquivos inexistentes, conexões de rede interrompidas ou entrada inválida do usuário).

Todas as exceções encontradas em Java seguem o padrão orientado a objetos, sendo objetos que herdam suas características da classe `Throwable`. A hierarquia de Throwable é dividida em duas grandes categorias:

- Error – erros graves da JVM, geralmente não tratados (ex: OutOfMemoryError);
- Exception – erros esperados e tratáveis em tempo de execução.

As exceções, por sua vez, são divididas em dois grupos principais:

- **Checked Exceptions (verificadas):** ocorrem em tempo de compilação, e devem ser obrigatoriamente tratadas ou declarados com `throws`.
- **Unchecked Exceptions (não verificadas):** ocorrem em tempo de execução *(runtime)*, e ão exigem tratamento explícito.

Vamos resumir as principais exceções numa tabela-resumo:

| Exceção | Descrição |
|---|---|
| ArithmeticException | Tentativa de operação matemática ilegal, como divisão por zero (/ 0) |
| ArrayIndexOutOfBoundsException | Acesso a índice inválido em um array (menor que 0 ou maior que o tamanho - 1) |
| ClassCastException | Tentativa de conversão de tipos incompatíveis (casting ilegal) |
| IllegalArgumentException | Argumento inválido passado a um método |
| IllegalStateException | Método chamado em estado inadequado do objeto |
| IndexOutOfBoundsException | Índice inválido em listas ou arrays |
| InputMismatchException | Entrada de dados de tipo incompatível (ex: ler int e digitar String) |
| NullPointerException | Tentativa de acessar método ou atributo de uma referência null |
| NumberFormatException | Conversão inválida de String para número (ex: Integer.parseInt("abc")) |
| UnsupportedOperationException | Operação não suportada por uma determinada coleção ou contexto |
| StringIndexOutOfBoundsException | Índice inválido em uma operação com String |
| IOException | Erro de entrada/saída (arquivo inexistente, falha ao ler) |

| | |
|---|---|
| FileNotFoundException | Arquivo não encontrado para leitura ou escrita |
| SQLException | Erros relacionados a operações SQL (conexão, comandos, etc.) |
| InterruptedException | Interrupção de uma thread em espera |
| StackOverflowError | Erro causado por chamadas recursivas infinitas que estouram a pilha da JVM |
| OutOfMemoryError | Erro causado por falta de memória disponível na JVM |
| NoSuchElementException | Elemento inexistente acessado em iteradores, scanners ou filas vazias |
| ConcurrentModificationException | Modificação indevida de uma coleção durante iteração |
| IllegalMonitorStateException | Uso incorreto de wait(), notify() sem possuir o monitor da thread |

Precisamos, de alguma forma, tratar essas exceções. O tratamento de exceções consiste no **uso de blocos de código específicos para lidar com situações anormais**, permitindo que o programa continue sua execução mesmo quando ocorrem erros. Isso garante que o código seja mais robusto, confiável e tolerante a falhas.

Java utiliza-se de um conjunto de palavras-chave para esse tratamento: `try`, `catch`, `finally`, `throw` e `throws`. Veja o bloco padrão:

```java
try {
    // bloco de código que pode lançar exceções
} catch (TipoDaExcecao e) {
    // tratamento da exceção
} finally {
    // bloco opcional que sempre será executado
}
```

Aqui:

- `try`: envolve o código que pode lançar exceções.
- `catch`: captura e trata a exceção especificada.
- `finally`: opcional, sempre é executado, com ou sem exceção, geralmente usado para liberar recursos (como arquivos, conexões, etc).

Veja uma aplicação mais “real”:

Java

```java
public class TratamentoExcecao {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Erro: divisão por zero.");
        } finally {
            System.out.println("Finalizando...");
        }
    }
}
```

```
Erro: divisão por zero.
Finalizando...
```

É possível que sejam elencados vários *catch*, para tratar diferentes tipos de erros.

Ainda, temos dois blocos interessantes: o `throw` e o `throws`. A palavra-chave `throw` permite **lançar uma exceção explicitamente**:

Java

```java
if (idade < 0) {
    throw new IllegalArgumentException("Idade não pode ser negativa.");
}
```

Já o `throws` é usado na assinatura de métodos. Ele indica que o método pode lançar exceções que devem ser tratadas por quem o chamar.

Java

```java
public void lerArquivo(String caminho) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(caminho));
    // ...
}
```

**(AOCP/TRF 2/2024)** Considerando o seguinte trecho de código, qual mensagem será impressa?

```
public class ErrorHandling {
   public static void main(String[] args) {
try {
   int[] numbers = new int[5];
   numbers[5] = 10;
   System.out.println(“Atribuição de valor efetuada com sucesso.”);
   }
catch (ArrayIndexOutOfBoundsException e) {
   System.out.println(“O índice do array ultrapassa o valor permitido.”);
  }
catch (Exception e) {
   System.out.println(“Ocorreu uma exceção de erro geral”);
  }
finally {
   System.out.println(“O ‘try catch’ finalizou.”);
   }
  }
}
```

a) Atribuição de valor efetuada com sucesso.
b) O índice do array ultrapassa o valor permitido.
c) Ocorreu uma exceção de erro geral.
d) O índice do array ultrapassa o valor permitido.\n O ‘try catch’ finalizou.
e) Ocorreu uma exceção de erro geral.\n O ‘try catch’ finalizou.

**Comentários:**

O código envolve o tratamento de exceções. Vamos decompô-lo, para analisar melhor o que acontece:

int[] numbers = new int[5]; → cria uma array de números com 5 *slots;*
numbers[5] = 10 → tentamos acessar o índice 5, mas numbers só tem os índices de 0 a 4, causando o erro ArrayIndexOutOfBoundsException.
O try é interrompido na linha que lança a exceção, portanto a mensagem "Atribuição de valor efetuada com sucesso." não chega a ser lida
Como a exceção é a ArrayIndexOutOfBoundsException, vamos no primeiro *catch*, retornando “O índice do array ultrapassa o valor perimitido”.
Como o *finally* sempre é executado, a frase “O ‘try catch’ finalizou” também vai ser impressa.

Portanto, a saída é “O índice do array ultrapassa o valor permitido. .\n O ‘try catch’ finalizou”. Correta a letra D. *(Gabarito: Letra D)*

# Métodos

Em Java, um **método é um bloco de código nomeado e reutilizável**, definido dentro de uma classe, que realiza uma tarefa específica. É basicamente uma função, que recebe um nome "diferente" por estar sendo utilizado no contexto da orientação a objetos. Esse método pode ou não receber parâmetros de entrada, assim como pode ou não retornar um valor.

A estrutura básica de um método é a que segue:

```
    modificadores tipoRetorno nomeDoMetodo(parâmetros) {
        // corpo do método
    }
```

Veja um exemplo simples:

Java

```java
public int somar(int a, int b) {
    return a + b;
}
```

Um método é composto por diferentes partes:

- **Modificadores** → Define o nível de acesso (public, private, protected) e outros (static, final, etc.)
- **tipoRetorno** → Tipo do valor que o método retorna (int, double, String, void, etc.)
- **nomeDoMetodo** → Nome identificador, segue convenção camelCase (calcularMedia, getNome)
- **parâmetros** → Lista (opcional) de variáveis de entrada com seus tipos
- **corpo** → Bloco entre { } com as instruções que o método executa

Aqui precisamos detalhar principalmente os modificadores e o tipo de retorno. Os **modificadores** são palavras-chave utilizadas para definir aspectos fundamentais do **comportamento e da visibilidade dos métodos**.

Eles controlam quem pode acessar, como pode acessar e em que contexto um método pode ser utilizado. São essenciais para a aplicação dos princípios de encapsulamento, modularidade, herança e projetos orientados a objetos em geral.

> *Orientação a objetos aplicado a Java é nosso próximo capítulo, então alguns assuntos podem parecer confusos aqui – caso sinta necessidade, veja primeiro a seção de Orientação a Objetos em Java e depois retorne a esse capítulo, embora eu acredite que não exista essa necessidade.*

São diferentes categorias de modificadores. Quanto à capacidade de acesso aos métodos, temos 4 classificações distintas, veja:

| Modificador | Descrição |
|---|---|

| | |
|---|---|
| public | O método **pode ser acessado de qualquer lugar** (outra classe, pacote, etc). |
| private | O método **só pode ser acessado dentro da mesma classe**. |
| protected | O método **pode ser acessado no mesmo pacote e por subclasses** (mesmo fora do pacote). |
| padrão (sem modificador) | O método é **acessível apenas dentro do mesmo pacote**. |

*Um pacote é o "nome do diretório" onde os arquivos .java e .class estão organizados, mas também é uma declaração dentro do código que indica a qual agrupamento lógico aquela classe pertence.*

Temos também o modificador `static`. Esse método pertence à classe em que é definido, e não à instância (objeto) – ou seja, ele pode ser chamado sem criar um objeto de classe, mas não pode acessar diretamente membros de instância (com métodos como `this` e `super`).

O modificador `final` faz com que métodos **não possam ser sobrescritos** (com `override`), garantindo que a lógica não seja alterada em heranças. O modificador `abstract` declara um **método abstrato**, sem corpo – ou sem `{}`. Métodos abstratos só podem ser definidos nas classes abstratas, e as subclasses devem obrigatoriamente implementar o método.

Resumindo:

| Modificador | Descrição |
|---|---|
| public | O método **pode ser acessado de qualquer lugar** (outra classe, pacote, etc). |
| private | O método **só pode ser acessado dentro da mesma classe**. |
| protected | O método **pode ser acessado no mesmo pacote e por subclasses** (mesmo fora do pacote). |
| static | O método **pertence à classe** (e não à instância), podendo ser chamado sem criar um objeto da classe. Não acessa membros de instância diretamente. |
| final | O método não pode ser sobrescrito (`overridden`) por subclasses, garantindo que sua implementação permaneça inalterada. |
| abstract | O método não possui corpo (implementação) e deve ser implementado por subclasses concretas. Só pode existir dentro de classes abstratas. |

Já os **tipos de retorno** são o tipo de dado que o método entrega de volta após sua execução. Ele aparece imediatamente antes do nome do método na sua declaração. Aqui quase não temos nenhuma novidade, com exceção do tipo específico para métodos: o tipo `void`. Ele indica que o método não retorna valor algum, usado quando o método executa ações mas não devolve nenhum resultado. Por exemplo:

```java
public void exibirErro() {
    System.out.println("Erro ao processar.");
}
```

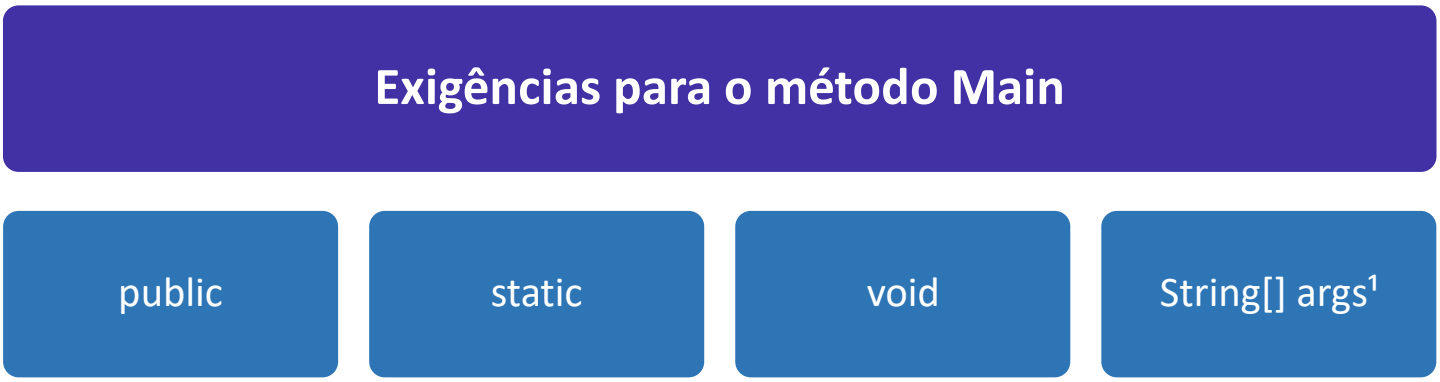
Veja que não temos a palavra-chave `RETURN`. Embora ela possa ser usada, seu uso se restringe ao encerramento do método. Ao contrário de métodos com algum tipo definido, que devem conter a expressão `RETURN` atrelada a algum resultado.

Outro elemento que você deve ver frequentemente é o parâmetro `String[] args`. Ele o parâmetro formal do método `main`, ponto de entrada padrão de qualquer aplicação Java, onde `String[]` define uma array de strings, e `args` é o nome da variável do parâmetro – apenas uma convenção padrão para definição da variável *main*.

O `args` basicamente serve para capturar os argumentos passados ao programa na linha de comando, permitindo que o programa seja parametrizado externamente. E esse padrão deve ser seguido no método main, que, além do parâmetro, deve ser:

- `public`
- `static`
- `void`

Resumindo:




[1] o nome do parâmetro `args` pode mudar, mas devemos ter como parâmetro uma array de strings, através do `String[]`



HORA DE PRATICAR!

**(AOCP/UFS/2024)** Considere o seguinte trecho de código Java:

```
public class Test {
  public static void main(String[] args) {
   String str = "Hello";
   changeStr(str);
   System.out.println(str);
  }

  public static void changeStr(String s) {
    s = "World";
  }
}
```

Qual será a saída do programa quando executado?

a) "Hello"
b) "World"
c) Null
d) Uma exceção é lançada.
e) "Hello World"

**Comentários:**

A questão apresenta dois métodos: o método main, que recebe a string str, inicializada com o valor “Hello” e, em seguida, o método changeStr(str), que irá chamar um outro método,

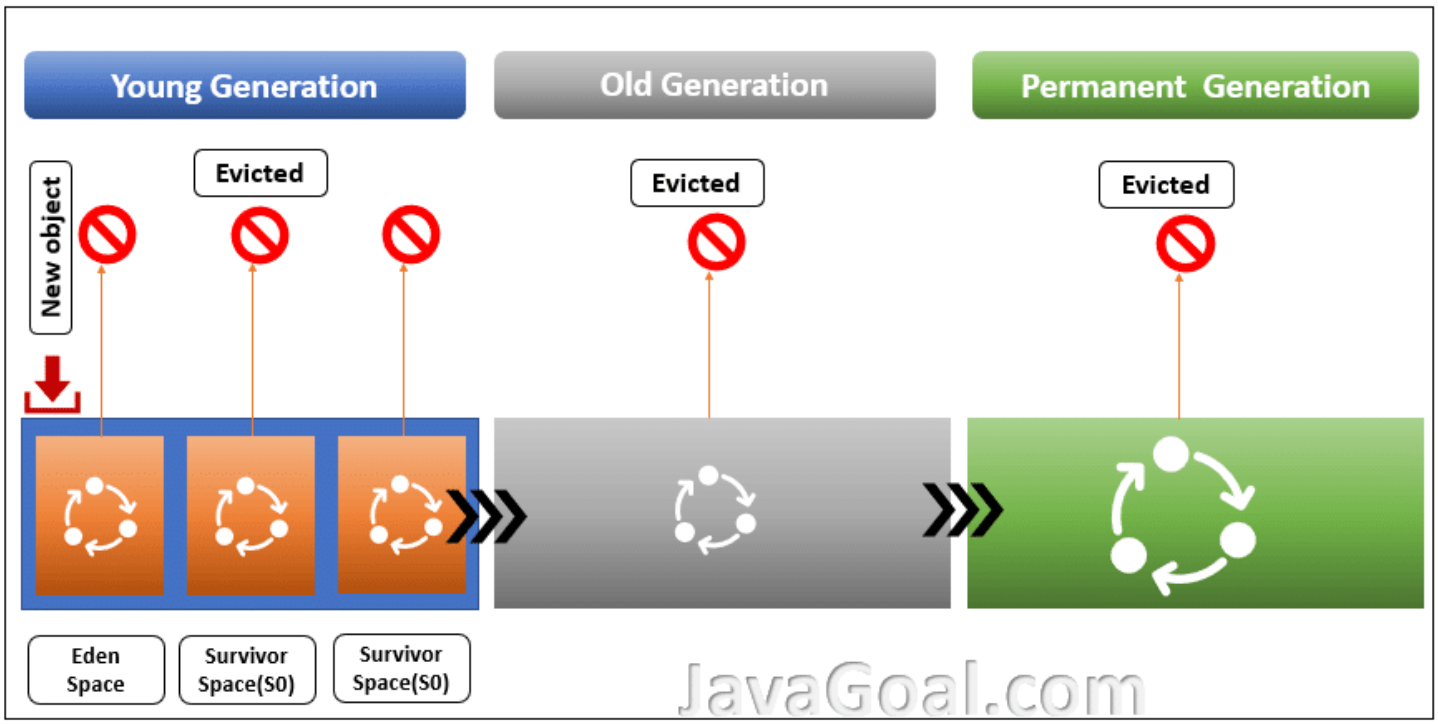
Esse método changeStr retorna a string s, com o conteúdo “World”. Então, num primeiro momento, você deve pensar “ah, então vamos substituir “Hello” por “World”, questão fácil – letra B.

Mas temos uma propriedade importante que você deve se lembrar: as **strings são imutáveis**. Então, ainda que exista uma nova declaração, o valor original não pode ser alterado – sendo assim, o retorno será o próprio “Hello” original. *(Gabarito: Letra A)*

# Administração de Memória

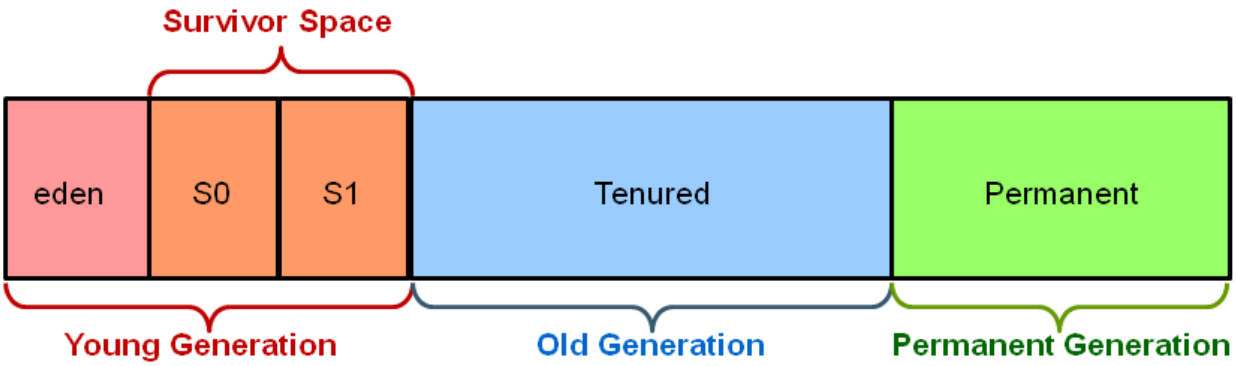
A **administração de memória** em Java é realizada de **forma automática**, robusta e segura por meio de um componente da máquina virtual chamado **Garbage Collector (GC)**. Esse mecanismo elimina a necessidade do programador gerenciar diretamente a alocação e liberação de memória, reduzindo o risco de erros como vazamentos (memory leaks), acesso a áreas inválidas ou uso de memória já liberada.

Ao executar um programa Java, a JVM organiza a memória em diferentes regiões, das quais a principal para alocação de objetos é chamada de **Heap.** A Heap é dividida em subáreas com propósitos distintos: *Young Generation* (geração jovem), onde objetos recém-criados são alocados, e *Old Generation* (geração antiga), que armazena objetos com maior tempo de vida. Há ainda a *Metaspace*, usada para armazenar metadados das classes carregadas pela JVM.




Quando um objeto é instanciado, a memória necessária é alocada na Young Generation, mais precisamente no espaço chamado Eden. Se ele sobreviver a sucessivas execuções do Garbage Collector, será promovido para as áreas Survivor e, posteriormente, para a Old Generation. Esse modelo é baseado na observação de que a maioria dos objetos em Java tem vida curta, sendo descartados logo após sua utilização.

O Garbage Collector identifica objetos que não são mais acessíveis pelo programa, ou seja, que não possuem mais referências ativas. Esses objetos são então considerados lixo e têm sua memória automaticamente liberada. A coleta pode ocorrer de forma parcial (como a Minor GC, focada na Young Generation) ou completa (Full GC, que percorre todas as regiões da Heap), e seu algoritmo pode variar conforme a implementação da JVM — entre os mais conhecidos estão o Serial GC, Parallel GC, G1 GC e o mais recente ZGC.




A administração de memória em Java também inclui o uso de **stack memory** (memória de pilha), onde são **armazenadas variáveis locais e chamadas de métodos**. A pilha é gerenciada de forma automática pela JVM e cresce ou encolhe de acordo com a profundidade da execução dos métodos, sendo liberada quando os métodos retornam.

**(QUADRIX/CFO/2025)** Em relação às linguagens Java e PHP, julgue o item a seguir.

A administração de memória em Java é automatizada pelo sistema de *garbage collection,* que identifica e libera a memória ocupada por objetos que não são mais referenciados, evitando-se vazamentos de memória.

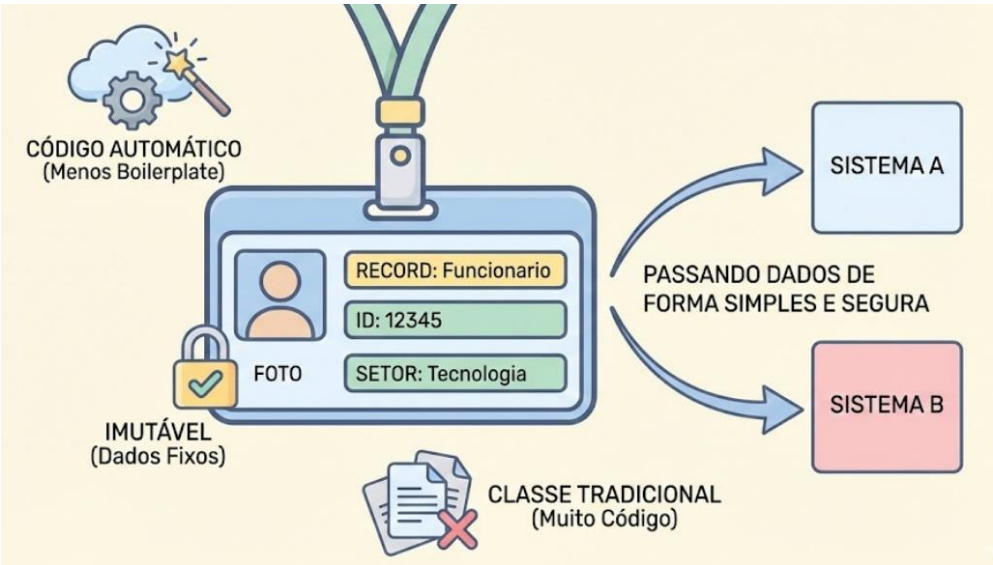
**Comentários:**

O item está correto. Em Java, a administração de memória é de fato automatizada por meio do Garbage Collector (GC), um componente da Java Virtual Machine (JVM) responsável por monitorar os objetos alocados na memória heap. Quando um objeto não é mais referenciado por nenhuma variável ativa, o GC o considera inacessível e pode automaticamente liberar a memória por ele ocupada.

Esse processo reduz significativamente a ocorrência de erros comuns em linguagens de gerenciamento manual, como vazamentos de memória, uso de ponteiros inválidos ou duplo free. Embora o GC não elimine completamente a possibilidade de memory leaks (caso o objeto permaneça referenciado indevidamente), ele atua de forma eficiente na reclamação automática da memória não utilizada, promovendo maior segurança e estabilidade na execução das aplicações Java. *(Gabarito: Certo)*

## Record Types

Imaginem que vocês estão desenvolvendo um sistema e precisam criar uma classe apenas para transportar dados temporários. É como se fosse um crachá corporativo, que só serve para mostrar seu nome e seu setor. No Java tradicional, criar essa classe gera um texto de código gigantesco e puramente burocrático com construtores, getters, equals e hashcode.




*Fonte: Prof. Vinicius Borges*

O Record type, introduzido de forma definitiva no Java 16, veio para matar essa burocracia de escrever um monte de código boilerplate. Ele é um tipo especial de classe focado estritamente em atuar como um portador de dados imutáveis. A máquina virtual do Java cria todo esse código repetitivo por baixo dos panos.

A sintaxe e assustadoramente simples, vejam:

```
public record Funcionario(String nome, String setor) {
}
```

Pronto. Com essa única linha, você tem uma classe completa. A regra de ouro para levar para a prova e a seguinte: todos os atributos declarados no cabeçalho do Record são finais, ou seja, imutáveis. Uma vez instanciado, você não pode mudar os valores. E para ler esses valores, você não usa os métodos com o prefixo tradicional, você usa o próprio nome do atributo, acessando como funcionario.nome() e funcionario.setor().

## Sealed Classes

Gente, pensem nas Sealed Classes, ou classes seladas, como a porta de uma festa VIP com uma lista restrita de convidados. A finalidade dessa funcionalidade é **restringir a herança na declaração da classe**, especificando de forma explícita quais classes podem herdar dela.




*Fonte: Prof. Vinicius Borges*

No Java antigo, a herança era polarizada. Se você criasse uma classe padrão, qualquer um poderia herdar. Se colocasse a palavra final, ninguém herdava. As classes seladas trazem o controle total. Você define na própria classe pai **quem são os herdeiros autorizados usando a palavra permits**.

Mas aqui vem a pegadinha de ouro que derrubou muita gente na prova da FGV para o MPU em 2025: se você declara uma classe como sealed, ela **obrigatoriamente precisa ter subclasses**. Se você criar uma classe selada e não definir nenhuma subclasse permitida para ela, o seu código vai dar **erro de compilação**. A classe selada nasceu para ser herdada, porém apenas por quem ela autorizar.

Outra regra rígida de compilação: toda classe que herda de uma classe selada é obrigada a declarar o seu próprio status de herança. Ela precisa usar a palavra final para fechar a herança, sealed para criar uma nova sub-lista VIP, ou non-sealed para abrir a herança novamente para o público geral.

Veja o exemplo da imagem em código:

```
public sealed class Documento permits Cpf, Passaporte {
}

public final class Cpf extends Documento {
}

public non-sealed class Passaporte extends Documento {
}
```

# Pattern matching

Galera, esse recurso otimiza a verificação de tipos e a conversão de variáveis. Pensem em como vocês checavam o tipo de um objeto antes. Primeiro verificava a identidade com **instanceof** e, se fosse verdadeiro, fazia o **casting manual** na linha de baixo. O Pattern Matching faz as duas coisas juntas: checa o tipo e já cria a variável convertida e pronta para uso.

Outro conceito avançado (cobrado pela Consulpam) é a combinação de Pattern Matching no switch com Sealed Classes. Como a classe selada restringe exatamente quem pode herdar dela, o compilador do Java tem certeza absoluta de todos os tipos possíveis que podem chegar naquele switch.

Isso se chama switch exaustivo. Nesses casos, o compilador garante que todos os casos foram tratados e você **nem precisa escrever a cláusula default**. Vejam como o código fica limpo:

```
Forma f = new Circulo(2.0);
String tipo = switch (f) {
    case Circulo c -> "Temos um círculo com raio " + c.raio;
    case Retangulo r -> "Temos um retângulo";
};
```

Notem que não tem default no código acima, e o Java compila perfeitamente porque a interface Forma era selada, permitindo unicamente as classes Circulo e Retangulo. Não existe outra possibilidade no universo do sistema, então o default se torna inútil.

# Virtual threads

Por fim, entramos em um tópico de altíssimo desempenho. No Java tradicional, cada thread de execução era amarrada diretamente a uma thread do sistema operacional do seu servidor. Isso era pesado e consumia muita memória.

As Virtual Threads são threads extremamente **leves** gerenciadas de forma **independente** pela própria máquina virtual Java, e não pelo sistema operacional. Se uma Virtual Thread precisa parar para esperar uma resposta de um banco de dados, a JVM guarda ela em uma área temporária e coloca outra Virtual Thread para usar o processador instantaneamente. Isso permite que um software rode **milhões de tarefas simultâneas** sem derrubar o servidor.

```
Thread.startVirtualThread(() -> {
    System.out.println("Executando de forma leve!");
});
```

Alguns detalhes importantes sobre o tópico com altas chances de cair em prova:

- Primeiro: Virtual Threads **não executam o código mais rápido**. Elas não melhoram a latência de uma tarefa individual. Uma operação matemática complexa vai demorar exatamente o mesmo tempo em uma thread normal ou virtual. O ganho real aqui é em **vazão**, conhecido como throughput. O sistema

não fica mais rápido para uma única pessoa, mas ele passa a suportar milhares de requisições ao mesmo tempo sem derrubar a aplicação por falta de memória.

- Segundo: você **não deve usar Pool de Threads para Virtual Threads**. No Java antigo, nós criávamos um pool com tamanho fixo porque criar threads era muito caro para o sistema, então nós as reciclávamos. Como as Virtual Threads são extremamente leves, a recomendação oficial da Oracle é a abordagem de **uma thread por requisição**. Você cria uma thread virtual nova para cada tarefa e o Garbage Collector a destrói quando terminar.

- Terceiro: em algumas situações muito específicas, como ao executar código dentro de um bloco com a palavra reservada **synchronized** ou ao executar métodos nativos fora do Java, a Virtual Thread pode ficar colada na Carrier Thread. Se ela bloquear nesse estado, **a thread do sistema operacional vai bloquear junto,** anulando toda a vantagem do recurso. As bancas amam cobrar que o uso inadequado de synchronized causa o **pinning** da Virtual Thread.

# OO em Java

A **Orientação a Objetos** (OO) é um paradigma de programação baseado na organização do código em objetos, que representam entidades do mundo real. Esses objetos são instâncias de classes, que definem seus atributos e comportamentos.

O Java segue esse paradigma de forma estrita, sendo uma das linguagens mais populares para desenvolvimento modular, reutilizável e escalável. Através da OO, conseguimos criar sistemas mais organizados e fáceis de manter, promovendo boas práticas como encapsulamento, herança, polimorfismo e abstração. Vamos ver como o Java implementa essas propriedades, e boas práticas.

## Classes

Em Java, **tudo gira em torno de classes**. Uma classe é um **modelo (blueprint)** que define as características e comportamentos de um objeto. Pense em uma classe como a planta de uma casa: ela descreve como a casa será construída, mas a casa em si só existe quando a planta é usada para construi-la.

Cada classe pode ter dois componentes: **atributos** e **métodos**. Atributos são características da classe, sua descrição (ou variáveis de instância), enquanto os métodos são funções que apontam o que a classe pode fazer.

Veja um exemplo de definição de classes:

Java

```java
// Definição da classe
public class Carro {
    // Atributos da classe (características)
    String modelo;
    int ano;
    double preco;

    // Método da classe (comportamento)
    void exibirDetalhes() {
        System.out.println("Modelo: " + modelo);
        System.out.println("Ano: " + ano);
        System.out.println("Preço: R$" + preco);
    }
}
```

Nesse código:

- **`public class Carro`** → Declaração da classe chamada Carro, com visibilidade pública.
- **`String modelo; int ano; double preco;`** → Atributos que armazenam informações sobre um carro.

- `void exibirDetalhes()` → Método que imprime as informações do carro.

Acontece que, por si só, uma classe não é "nada". Ela é apenas uma definição genérica do que pode ser implementado. Para essa implementação, precisamos criar **objetos**. Os objetos são caracterizados pelo uso da palavra-chave `new` – como em `new Carro()`.

Java

```java
public class TesteCarro {
    public static void main(String[] args) {
        // Criando um objeto da classe Carro
        Carro meuCarro = new Carro();

        // Definindo valores para os atributos
        meuCarro.modelo = "Civic";
        meuCarro.ano = 2022;
        meuCarro.preco = 150000.00;

        // Chamando o método da classe
        meuCarro.exibirDetalhes();
    }
}
```

```
Modelo: Civic
Ano: 2022
Preço: R$150000.0
```

Assim:

- Criamos um objeto `meuCarro` da classe Carro.
- Definimos valores para os atributos do carro.
- Chamamos o método `exibirDetalhes()`, que imprime os dados do carro.

Veremos mais à frente quando analisarmos heranças, mas cada classe traz métodos para acessar e interagir com seus próprios métodos ou de suas superclasses:

| Método | Descrição |
|---|---|
| super() | Chama o construtor da superclasse |
| super.metodo() | Chama um método da superclasse |
| this() | Chama outro construtor da mesma classe |
| instanceof | Verifica se um objeto pertence a uma classe |

# Interfaces

Uma **interface** é uma estrutura que define **apenas assinaturas de métodos**, sem implementar sua lógica. As classes que "assinam o contrato" (ou seja, implementam a interface) são obrigadas a fornecer implementações concretas para esses métodos.

Veja um exemplo para definição da interface `Animal` e, em seguida, sua implementação através da classe `Cachorro`.

Java

```java
interface Animal {
    void fazerSom();  // Método abstrato (sem corpo)
}

class Cachorro implements Animal {
    public void fazerSom() {
        System.out.println("Au au!");
    }
}
```

# Herança

Herança é o mecanismo pelo qual uma classe (subclasse ou classe derivada) herda os atributos e métodos de outra classe (superclasse ou classe base). Em Java, para implementarmos a herança, usamos a palavra-chave `extends`.

No código exemplo abaixo, definimos a superclasse `Animal`, veja:

Java

```java
class Animal {
    String nome;

    void fazerSom() {
        System.out.println("Som genérico de animal");
    }
}
```

Podemos estender essa classe para criar subclasses, a partir da diretiva `extends`. Veja:

Java

```java
class Cachorro extends Animal {
    void fazerSom() {
        System.out.println("Au au!");
    }
}

public class Main {
    public static void main(String[] args) {
        Cachorro meuCachorro = new Cachorro();
        meuCachorro.nome = "Rex";
        meuCachorro.fazerSom();  // Saída: Au au!
    }
}
```

Veja que já implementamos o objeto na classe `Main`, e, além disso, **sobrescrevemos o método `fazerSom()`**, trazendo um novo retorno – "Au au!". Só precisamos ter atenção às definições dos métodos, já que métodos do tipo `private` não são herdados.

Outra propriedade que temos que ficar atentos é a palavra-chave `final`. Quando aplicada a classes, não permite que eles sejam estendidos. Já quando aplicados a métodos, não permite que eles sejam sobrescritos.

**(FGV/TJ AP/2024)** Analise o código Java a seguir, no qual foi ocultada a segunda linha de código.

```
abstract class X {
   ---
   public void exibe() {
      System.out.println("Concluído");
   }
}
class Z extends X {
  public void X1(String P) {
     System.out.println(P);
  }
}
class Main {
  public static void main(String[ ] args) {
     Z objZ = new Z( );
     objZ.X1("OK");
     objZ.exibe( );
  }
}
```

O trecho que substitui corretamente a linha tracejada é:

a) public abstract void X1(String P);

b) public static void X1(String P);
c) public String X1(String P);
d) public void X1(String P);
e) public X1(String P);

**Comentários:**

Temos uma estrutura orientada a objetos. Estamos definindo uma classe abstrata X, que posteriormente é estendidade pela subclasse Z e implementada pela classe Main (isso é, com objeto criado a partir dela). Precisamos preencher o campo ---, que representa a chamada do método geral da classe abstrata. Por se tratar de uma classe abstrata, devemos também implementar um método abstrato, através do comando public abstract void X1(String P), já que podemos ter retornos void, o método é público e os parâmetros são *strings*. *(Gabarito: Letra A)*

**(FUNDATEC/IFC/2023)** Considere o código abaixo, em linguagem Java:

```
abstract class Animal {

   public void andar() {
      System.out.println("Animal andando");
   }
}

class Cachorro extends Animal {

   public void andar() {
      System.out.println("Cachorro andando");
   }
}

class Gato extends Animal {

   public void andar() {
      System.out.println("Gato andando");
   }
}

public class Main {

   public static void main(String[] args) {
      Animal cachorro = new Cachorro();
      Gato gato = new Gato();
      cachorro.andar();
      cachorro = gato;
      cachorro.andar();
      Animal animal = new Animal();
      animal.andar();
```



02648216103 - HENRIQUE AMORIM BARRETO

```
    }
}
```

Qual a saída do programa?

a) Cachorro andando, Animal andando, Gato andando.
b) Cachorro andando, Gato andando, Animal andando.
c) Cachorro andando, Cachorro andando, Animal andando.
d) Gato andando, Cachorro andando, Animal andando.
e) Não compila o código.

**Comentários:**

Temos uma classe abstrata Animal, que é estendida por Cachorro e Gato, ambos implementando o método andar() do polimorfismo dinâmico. Entretanto, temos uma classe abstrata (Animal), que implementa o método andar(). No método main, temos a instanciação desse método com “animal.Andar()”, o que gera um erro de compilação: não podemos instanciar diretamente classes abstratas, ou seja, não podemos criar objetos dela diretamente através do método new.

Uma classe abstrata serve como modelo ou "template" para outras classes (subclasses). Seu objetivo principal é definir métodos e/ou comportamentos que devem ser herdados ou implementados por suas subclasses. Porém, nunca pode ser usada diretamente para criar objetos, porque ela representa um conceito genérico demais. Por esse motivo, não conseguiremos compilar o código. *(Gabarito: Letra E)*

## Encapsulamento

Encapsulamento é o conceito de **esconder os dados internos** (atributos) de uma classe, fornecendo acesso controlado por meio de métodos específicos (getters e setters). O objetivo é evitar que outras classes manipulem diretamente os atributos, reduzindo erros e garantindo consistência.

O encapsulamento em Java envolve duas etapas principais:

- Declarar atributos como privados (`private`).
- Criar métodos públicos para acessar e modificar esses atributos (`getters` e `setters`).

Veja um exemplo:

Java

```java
public class Pessoa {
    // Atributo privado (encapsulados)
    private String nome;

    // Método Getter para atributo nome(acesso ao valor)
    public String getNome() {
        return nome;
    }

    // Método Setter (modifica o valor com validação)
    public void setNome(String nome) {
        if (nome != null && nome.length() >= 3) {
            this.nome = nome;
        } else {
            throw new IllegalArgumentException("Nome inválido!");
        }
    }
}
```

## Polimorfismo

**Polimorfismo** é a capacidade que um objeto tem de ser referenciado por variáveis de diferentes tipos, assumindo comportamentos específicos dependendo da instância real do objeto. Por exemplo, um objeto do tipo Gato ou Cachorro pode ser tratado como do tipo mais genérico Animal, mas cada um mantém seu comportamento específico. Existem dois tipos de polimorfismo possíveis em Java:

- Polimorfismo de **sobrecarga**, ou **estático**
- Polimorfismo de **sobrescrita**, ou **dinâmico**

O polimorfismo de sobrecarga acontece quando métodos têm o mesmo nome, porém assinaturas diferentes (parâmetros diferentes). A escolha do método ocorre em tempo de compilação.

Java

```java
class Calculadora {
    int somar(int a, int b) {
        return a + b;
    }

    double somar(double a, double b) {
        return a + b;
    }
}
```

Já o polimorfismo de sobrescrita ocorre quando uma classe filha (subclasse) implementa um método com mesma assinatura de uma classe pai (superclasse). A decisão sobre qual método será chamado é feita em tempo de execução, com base no tipo real do objeto.

Java

```java
class Cachorro extends Animal {
    void fazerSom() {
        System.out.println("Au au!");
    }
}

class Gato extends Animal {
    void fazerSom() {
        System.out.println("Miau!");
    }
}
```

# QUESTÕES COMENTADAS

## Teoria do Java

1. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática)**

Com relação às linguagens de programação Java e PHP, julgue o item a seguir.
Na linguagem Java, os tipos int, char e long são exemplos de dados elementares, e os tipos matrizes e vetores são dados estruturados da linguagem.

**Comentários**:
Gente, essa aqui é fundamental. O Java separa o que é básico do que é complexo. Os tipos primitivos como int, char e long guardam o valor direto na memória e são dados elementares. Já os arrays (matrizes e vetores) são tratados como objetos, ou seja, são estruturas mais complexas que apontam para endereços de memória, sendo corretamente classificados como dados estruturados.

**Gabarito**: C

2. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática), adaptada**

Julgue o próximo item, relativo a controle de dados, compilação e engenharia de software.
Os programas em Java podem ser tanto interpretados quanto compilados.

**Comentários**:
Cuidado com a pegadinha. Apesar de popularmente ser conhecido como uma linguagem compilada, o Java é, tecnicamente, uma linguagem híbrida. O seu código não é simplesmente compilado para linguagem de máquina nem apenas interpretado do zero. Ele é primeiro compilado para um formato intermediário chamado bytecode e, depois, esse bytecode é interpretado ou compilado em tempo de execução (JIT) pela Máquina Virtual Java (JVM).

**Gabarito**: C

3. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Analista Judiciário - Área Apoio Especializado - Especialidade: Analista de Sistemas)**

Acerca das tecnologias que atendem a PDPJ-Br, julgue o próximo item.

Em Java, uma subclasse herda os membros, campos e métodos de sua superclasse direta, podendo acrescentar novos membros, desde que mantenha intactos os membros herdados.

**Comentários**:
Quando uma classe herda de outra, ela recebe automaticamente tudo o que a "mãe" (superclasse) tem, como campos e métodos. O que torna a questão errada é “desde que mantenha intactos os membros herdados”, pois as classes filhas podem, sim, sobrescrever as funções originais das classes pai.

**Gabarito**: E

4. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Analista Judiciário - Área Apoio Especializado - Especialidade: Analista de Sistemas)**

Acerca das tecnologias que atendem a PDPJ-Br, julgue o próximo item.

As interfaces do Java permitem que objetos de classes não relacionadas sejam processados de forma polimórfica.

**Comentários**:
Perfeito! Em Java, interfaces definem um contrato (métodos) que pode ser implementado por classes não relacionadas entre si (ou seja, que não pertencem à mesma hierarquia de herança).

Quando diferentes classes implementam a mesma interface, seus objetos podem ser tratados pelo tipo da interface, permitindo o polimorfismo. Assim, o mesmo método pode ser chamado em objetos de classes distintas, desde que todos implementem aquela interface.

**Gabarito**: C

5. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Técnico Judiciário - Área Apoio Especializado - Especialidade: Programação de Sistemas)**

Julgue o item seguinte, relativos a Java.
Em Java, as classes possuem herança múltipla, de modo que cada classe pode ser derivada de mais de uma superclasse direta.

**Comentários**:
Gente, essa aqui você não pode errar de jeito nenhum. **Java NÃO suporta herança múltipla de classes**. Uma classe só pode ter uma "mãe" direta (extends). O que o Java permite é que uma classe implemente várias interfaces, mas isso não é a mesma coisa que herança múltipla de classes, que é o que o item afirmou.

**Gabarito**: E

6. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Técnico Judiciário - Área Apoio Especializado - Especialidade: Programação de Sistemas)**

Julgue o item seguinte, relativos a Java.
O polimorfismo em Java permite processar objetos que derivam da mesma superclasse, direta ou indiretamente; cada objeto pode executar ações diferentes a partir da mesma chamada.

**Comentários**:
Galera, essa é a definição perfeita de polimorfismo. Imagine que você tem uma classe Animal com o método emitirSom(). Se você chamar esse método em um Cachorro, ele late; se chamar em um Gato, ele mia. É a mesma chamada de método, mas cada objeto executa sua própria ação dependendo de quem ele realmente é na memória.

**Gabarito**: C

7. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Analista de Sistemas)**

A respeito de linguagens de programação, julgue o próximo item.
A linguagem Java suporta arrays multidimensionais de forma direta, sem a necessidade de manipulação em cascata de arrays unidimensionais ou bidimensionais.

**Comentários**:
O erro aqui é a palavra "direta". Java não lida com matrizes como um bloco único e direto. Ele faz uma manipulação em cascata: uma variável aponta para um array, e cada posição desse array aponta para outro array. É uma estrutura de "array de arrays", e não uma matriz multidimensional nativa e contígua como em outras linguagens.

**Gabarito**: E

8. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Analista de Sistemas)**

A respeito de linguagens de programação, julgue o próximo item.
Os bytecodes gerados pelo compilador Java independem da plataforma em que são executados, diferentemente das instruções em linguagem de máquina.

**Comentários**:
Pessoal, esse é o coração do lema "escreva uma vez, rode em qualquer lugar". O compilador gera o bytecode (.class), que é uma linguagem neutra. Ele não sabe se vai rodar no Windows, Linux ou Mac. Quem se vira com as particularidades do sistema operacional é a JVM de cada plataforma, garantindo que o bytecode seja independente.

**Gabarito**: C

9. **(Ano: 2025 Banca: CESPE / CEBRASPE Órgão: Polícia Federal Prova: CESPE / CEBRASPE - 2025 - Polícia Federal - Perito Criminal Federal - Área 3: Informática Forense)**

Julgue os próximos itens, a respeito das características e das aplicações básicas das linguagens Java e JavaScript.
Devido à compilação do código-fonte em bytecode, que é executado pela máquina virtual Java (JVM), os programas Java podem ser executados em diversos sistemas operacionais, sem a necessidade de recompilação.

**Comentários**:
A beleza da JVM é justamente essa. Você compila o seu programa uma única vez no seu computador e pode levar o arquivo gerado para qualquer servidor ou dispositivo que tenha uma JVM instalada, sem precisar tocar no código-fonte ou recompilar nada.

**Gabarito**: C

10. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática)**

Julgue o próximo item, relativo a controle de dados, compilação e engenharia de software.

Na linguagem Java, existe um mecanismo próprio de gerenciamento de memória denominado garbage collector.

**Comentários**:
Aqui cabe mencionar uma impressão que a banca não considerou. Embora o Garbage Collector (Coletor de Lixo) seja essencial no ecossistema Java, ele é, tecnicamente, uma característica da Máquina Virtual Java (JVM) e não necessariamente uma regra intrínseca da sintaxe da "linguagem" em si.

Outrossim, a banca considerou “linguagem Java” como um termo amplo e manteve o gabarito como correto.

**Gabarito**: C

**11. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: TRT - 10ª REGIÃO (DF e TO) Prova: CESPE / CEBRASPE - 2025 - TRT - 10ª REGIÃO (DF e TO) - Analista Judiciário – Área: Apoio Especializado – Especialidade: Tecnologia da Informação)**
Acerca de desenvolvimento de sistemas, julgue o próximo item.
Em Java, uma das maneiras de se implementar o polimorfismo é a partir de interfaces que permitem o tratamento de objetos de classes diferentes de forma uniforme.

**Comentários**:
Perfeito! Em Java, o polimorfismo pode ser implementado por meio de interfaces. Quando classes diferentes implementam a mesma interface, seus objetos podem ser tratados de forma uniforme, isto é, por meio do tipo da interface, independentemente da classe concreta.

Isso permite que um mesmo código trabalhe com objetos de classes distintas, desde que todos cumpram o contrato definido pela interface, caracterizando o polimorfismo.

**Gabarito**: C

**12. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: SUSEP Prova: CESPE / CEBRASPE - 2025 - SUSEP - Analista Técnico: - Área: Tecnologia da Informação e Ciência de Dados)**
Julgue o item que se segue, referente a HTML, CSS e JavaScript, Python e Java.
No Java, versão 21, o bloqueio de uma virtual thread, por exemplo, em uma operação de I/O, bloqueia a platform thread subjacente.

**Comentários**:
Galera, no Java 21, as virtual threads (introduzidas pelo Project Loom) não bloqueiam a platform thread subjacente quando ficam bloqueadas em operações como I/O. Quando uma virtual thread realiza uma operação bloqueante (ex.: leitura de arquivo ou socket), o que acontece é:

- a virtual thread é suspensa (parked);
- a platform thread (carrier thread) é liberada para executar outras virtual threads.

Esse comportamento é justamente o principal benefício das virtual threads: **alta escalabilidade**, permitindo milhares ou milhões de threads concorrentes sem desperdiçar threads do sistema operacional.

**Gabarito**: E

**13. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-GO Prova: CESPE / CEBRASPE - 2024 - MPE-GO - Analista em Informática)**

Julgue o item seguinte, com base nos conceitos de linguagem de programação Ruby e Java.
Ao se utilizar a estrutura try-catch-finally para tratar certos tipos de erros em um programa Java, as instruções do bloco finally são úteis para realizar operações, independentemente do resultado do bloco try.

**Comentários**:
Galera, o bloco finally foi desenhado para conter códigos de limpeza, como fechar uma conexão com banco de dados ou um arquivo, e a regra de ouro é: **ele sempre executa**. Não importa se o código no try rodou ou se caiu em um catch, o finally será acionado de qualquer forma.

**Gabarito**: C

**14. (Ano: 2025 Banca: IMPARH Órgão: CGM de Fortaleza - CE Prova: IMPARH - 2025 - CGM de Fortaleza - CE - Auditor de Controle Interno - Área 2 (Ciências da Computação))**

Sobre os conceitos relacionados à orientação a objetos na linguagem de programação Java, marque a opção correta.
A) Java permite herança múltipla de classes diretamente.
B) Todos os métodos de uma classe abstrata devem ser abstratos.
C) Classes em Java podem implementar várias interfaces.
D) Uma classe com métodos e variáveis públicas não pode ser encapsulada.

**Comentários:**

Vamos analisar as alternativas.

a) Errado. Java não permite herança múltipla de classes diretamente. Isso é justamente para evitar o problema da ambiguidade (conflitos) que surgem com herança múltipla.
b) Errado. Uma classe abstrata pode conter métodos abstratos e métodos concretos (com implementação).
c) Certo. Java permite a implementação múltipla de interfaces, o que viabiliza uma forma de herança múltipla comportamental.
d) Errado. Uma classe com métodos e atributos públicos pode, sim, ser encapsulada.

Portanto, correta a letra C.

**Gabarito:** Letra C

---

**15. (Ano: 2025 Banca: FGV Órgão: PC-MG Prova: FGV - 2025 - PC-MG - Perito Criminal - Área II)**

Considere as características fundamentais da linguagem Java e seus recursos no desenvolvimento de aplicações.
Com base nos conceitos básicos de orientação a objetos, em relação ao uso de classes e objetos no Java, assinale a afirmativa correta.
A) Em Java, uma classe pode ser instanciada apenas se tiver um construtor público ou um construtor com especificadores de acesso adequados.
B) O modificador public impede que um método ou variável de instância seja acessado fora da sua própria classe.



02648216103 - HENRIQUE AMORIM BARRETO

C) Uma classe semi-abstrata não pode ser instanciada diretamente, mas pode conter métodos implementados e não implementados.
D) O conceito de polimorfismo permite que um objeto se comporte de forma diferente dependendo do tipo da referência usada para acessá-lo.
E) O uso da palavra-chave super é utilizado para acessar membros da classe pai em classes que herdam de outra classe.

**Comentários:**

Vamos às alternativas.

a) Errado. A classe pode ser instanciada mesmo que não tenhamos um construtor público, não há essa necessidade.
b) Errado. O modificador public permite acesso total ao método. O modificador que impede o acesso fora da classe é o private.
c) Errado. Java não possui o termo "classe semi-abstrata". O termo correto é apenas classe abstrata.
d) Errado. O polimorfismo permite que objetos se comportem de maneira diferente dependendo do tipo real do objeto criado (não do tipo da referência usada para acessá-lo).
e) Certo. super é usado para chamar métodos e construtores da classe pai (superclasse). Amplamente utilizado para acessar comportamentos originais após sobrescrita de métodos.

Portanto, correta a letra E.

**Gabarito:** Letra E

---

**16. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

O Java oferece uma forte tipagem estática, o que significa que os tipos de dados de variáveis devem ser explicitamente definidos em tempo de compilação, garantindo maior segurança e detecção precoce de erros.

**Comentários:**

Correto. Java é uma linguagem de tipagem estática e forte, o que significa que os tipos de variáveis são verificados em tempo de compilação e não podem mudar em tempo de execução, além de exigir compatibilidade explícita entre tipos. Isso aumenta a segurança do código e permite a detecção precoce de erros, ainda na fase de compilação.

**Gabarito:** Certo

---

**17. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.
Em Java, as variáveis de tipo primitivo não consomem memória, pois são apenas referências a objetos armazenados na heap.

**Comentários:**

A questão descreve, na realidade, as variáveis de referência. Os tipos primitivos criam novos espaços em memória.

**Gabarito:** Errado

---

**18. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

Em Java, os tipos de dados primitivos são passados por referência, permitindo que suas mudanças dentro de um método afetem diretamente a variável original.

**Comentários:**

Em Java, os tipos de dados primitivos (como int, double, char, boolean, entre outros) são passados por valor, e não por referência. Isso significa que, ao passar um valor primitivo como argumento para um método, o que é transmitido é uma cópia do valor da variável original. Portanto, qualquer alteração feita dentro do método não afeta a variável original fora do método.

**Gabarito:** Errado

---

**19. (Ano: 2024 Banca: IBFC Órgão: TRF - 5ª REGIÃO Prova: IBFC - 2024 - TRF - 5ª REGIÃO - Técnico Judiciário - Área de Apoio Especializado - Especialidade Desenvolvimento de Sistemas da Informação)**

A linguagem Java é amplamente utilizada mundialmente, devido a sua robustez e consolidação na área da computação. Diante do exposto, analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).

( ) A Máquina Virtual Java (JVM) permite que os programas Java sejam executados em diferentes plataformas, proporcionando portabilidade.
( ) Java não suporta herança, um dos princípios fundamentais da programação orientada a objetos.
( ) Java suporta coleta de lixo, o que significa que o programador não precisa gerenciar explicitamente a alocação e desalocação de memória.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

a) V - V - V
b) V - F - V
c) V - F - F
d) F - V - V



02648216103 - HENRIQUE AMORIM BARRETO

**Comentários:**

Vamos analisar os itens.

*( ) A Máquina Virtual Java (JVM) permite que os programas Java sejam executados em diferentes plataformas, proporcionando portabilidade.*
Verdadeiro. O código Java é compilado em bytecode, que é interpretado pela JVM. Isso permite que o mesmo programa seja executado em qualquer sistema operacional que tenha uma JVM compatível, garantindo a característica "write once, run anywhere".

*( ) Java não suporta herança, um dos princípios fundamentais da programação orientada a objetos.*
Falso. Java suporta herança sim, por meio da palavra-chave extends para herança de classes e implements para interfaces. O que Java não suporta é herança múltipla de classes, mas isso não invalida o suporte à herança como princípio.

*( ) Java suporta coleta de lixo, o que significa que o programador não precisa gerenciar explicitamente a alocação e desalocação de memória.*
Verdadeiro. Java possui Garbage Collector, que gerencia automaticamente a liberação de memória ocupada por objetos que não são mais referenciados.

Portanto, ficamos com V-F-V.

**Gabarito:** Letra B

---

**20. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: FINEP Prova: CESPE / CEBRASPE - 2024 - FINEP - Analista - Tecnologia da Informação - Software)**

Assinale a opção em que é apresentado o principal método usado na linguagem Java para iniciar a execução de um programa.

a) execute( )
b) run( )
c) start( )
d) begin( )
e) main( )

**Comentários:**

O método mais comum e presente em toda classe Java é o `main(String[] args)`. Ele que demarca o início do programa principal dentro da aplicação Java e será o primeiro método a ser chamado pela JVM.

**Gabarito:** Letra E

---

**21. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: FINEP Prova: CESPE / CEBRASPE - 2024 - FINEP - Analista - Tecnologia da Informação - Software)**

Assinale a opção em que é indicado o modificador de acesso em Java que pode permitir que os membros de uma classe sejam acessados apenas por classes do mesmo pacote.

a) package-private
b) internal
c) static
d) public
e) private

**Comentários:**

O modificador de acesso package-private (também conhecido como default access) é o comportamento padrão em Java quando nenhum modificador é explicitamente declarado antes de atributos, métodos ou classes (não públicas). Nesse caso, o membro só pode ser acessado por outras classes que estejam no mesmo pacote.

**Gabarito:** Letra A

---

**22. (Ano: 2024 Banca: FUNCERN Órgão: IF-RN Prova: FUNCERN - 2024 - IF-RN - Técnico de Tecnologia da Informação)**

Um item de dado de programa pode ser definido como uma constante ou variável. As constantes, por exemplo, possuem um valor predefinido e que não mudam durante a execução do programa. Já as variáveis são um item de dado cujo valor pode ser alterado durante a execução do programa. Nesse contexto, a palavra-chave usada na linguagem Java para declarar uma constante é

a) const.
b) final.
c) let.
d) protected.

**Comentários:**

Java não tem constantes "nativas". Para declarar uma, usamos a palavra-chave `final`. Ela declara que o valor da variável é final e não pode ser modificado.

**Gabarito:** Letra B

---

**23. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

Na linguagem Java, para impedir que um método criado em uma classe J não possa ser sobrescrito por alguma outra classe K que herde de J, deve-se usar a palavra

a) extends
b) final
c) implements
d) super
e) void

**Comentários:**

Outra questão cobrando o modificador `final`. Como o valor é final, não podemos sobrescrevê-lo.

**Gabarito:** Letra B

---

**24. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

Em um projeto desenvolvido na linguagem Java, considere um método P que chama um método Q. Existe uma palavra que, quando colocada na assinatura do método Q, faz com que, caso ocorra uma exceção nesse método, o tratamento dessa exceção seja da responsabilidade do método P.

Qual é essa palavra?

a) catch
b) Exception
c) finally
d) throws
e) try

**Comentários:**

Para lançarmos uma exceção, usamos a palavra-chave `throws`. Ela é usada na assinatura de um método para indicar que ele pode lançar uma exceção e que o tratamento será responsabilidade do método chamador.

**Gabarito:** Letra D

---

**25. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

É uma característica da linguagem Java conseguir representar a

a) codificação de métodos completos em interfaces
b) herança múltipla de interface
c) herança múltipla de classe
d) instanciação de classes abstratas
e) instanciação de interfaces

**Comentários:**

Vamos analisar as alternativas.

a) Errado. Antes do Java 8, interfaces não podiam conter métodos completos. A partir do Java 8, isso é permitido com métodos default, mas não é a principal característica associada à linguagem.
b) Certo. A linguagem Java não permite herança múltipla de classes, mas permite herança múltipla de interfaces. Isso significa que uma classe pode implementar várias interfaces ao mesmo tempo, e uma interface pode estender várias outras interfaces.
c) Errado. Java não permite herança múltipla de classes.

d) Errado. Classes abstratas não podem ser instanciadas diretamente.
e) Errado. Interfaces não podem ser instanciadas diretamente.

Portanto, correta a letra B.

**Gabarito:** Letra B

# Códigos em Java

**26. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-TO Prova: CESPE / CEBRASPE - 2024 - MPE-TO - Analista Ministerial Especializado - Área de Atuação: Análise de Sistemas)**

Em relação à linguagem de programação Java, julgue o item a seguir.
Durante a execução do código a seguir, ocorrerá uma exceção ArithmeticException.

```java
public class Main {
    public static void main(String[] args) {
        int numero = 10;
        int divisor = 0;

        try {
            int resultado = numero / divisor;
            System.out.println("Resultado: " + resultado);
        } catch (ArithmeticException e) {
            e.printStackTrace();
        }
    }
}
```

**Comentários**:
No mundo dos números inteiros em Java, a divisão por zero é um "pecado" capital. Quando o código tenta dividir 10 / 0, a JVM imediatamente interrompe o fluxo e lança uma `ArithmeticException`. Como o código tem um bloco catch preparado exatamente para essa exceção, ela será capturada e tratada.

Cabe destacar que se a divisão de zero fosse do tipo float, o erro seria diferente: `Infinity`, `-Infinity` ou `NaN`, a depender da divisão.

**Gabarito**: C

**27. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: EMBRAPA Prova: CESPE / CEBRASPE - 2025 - EMBRAPA - Analista – Área: Gestão da Informação – Subárea: Engenharia de Infraestrutura e Tecnologia da Informação)**

Julgue o seguinte item, referente à programação e estruturas de dados.

O código a seguir, em Java, retornará `Tecnológica` como resultado de sua execução.

```java
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> embrapa = new ArrayList<String>();

        embrapa.add("Ageitec");
        embrapa.add("Agência");
        embrapa.add("Embrapa");
```

```
        embrapa.add("de Informação");
        embrapa.add("Tecnológica");

        int a = 1;

        for (int i = 0; i < embrapa.size() - 2; ++i) {
            a = a + 2;
        }
        System.out.println(embrapa.get(a - 3));
    }
}
```

**Comentários**:
Vamos fazer o rastreio desse código, gente. O ArrayList tem 5 elementos (tamanho 5). O laço for roda enquanto i < 3 (5 - 2), ou seja, ele executa para i = 0, 1 e 2 (3 vezes total). A variável a começa em 1 e ganha +2 em cada volta.

- Laço 1: a = 3
- Laço 2: a = 5
- Laço 3: a = 7
- No final, o código pede embrapa.get(7 - 3), que é o get(4). O índice 4 da lista é justamente "Tecnológica".

**Gabarito**: C

**28. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: FUB Prova: CESPE / CEBRASPE - 2025 - FUB - Técnico de Tecnologia da Informação)**

No que se refere à linguagem de programação Java, julgue o item subsecutivo.

Considere a seguinte classe **Estudante**:

```
public class Estudante {
    private String nome;
    private int matricula;

    public Estudante(String nome, int matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }
}
```

Considere, também, o seguinte trecho de código, que visa cadastrar e exibir os dados de um estudante:

```
public class Main {
    public static void main(String[] args) {
        Estudante e = new Estudante("Carlos", 12345);

        System.out.println("Nome: " + e.nome);
```

```
        System.out.println("Matrícula: " + e.matricula);
    }
}
```

A execução dos trechos de código precedentes criará um objeto da classe *Estudante* com os dados informados e exibirá as informações na tela sem apresentar erro de compilação.

**Comentários**:
Galera, aqui está a "casca de banana" clássica de orientação a objetos. Note que os atributos nome e matricula na classe Estudante foram declarados como **private**. Isso significa que eles estão trancados dentro da classe. Quando a classe Main tenta acessar e.nome e e.matricula diretamente, o compilador vai barrar na hora por **falta de visibilidade**. Para funcionar, precisaríamos de métodos public (getters).

**Gabarito**: E

**29. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: TRF - 6ª REGIÃO Prova: CESPE / CEBRASPE - 2025 - TRF - 6ª REGIÃO - Analista Judiciário – Área: Apoio Especializado – Especialidade: Análise de Dados)**
Julgue o próximo item, relativo a Java.

```
import java.io.*;
class Easy {
    public static void main(String[] args){
        int x[] = {1, 3, 7, 22, 51};
        for (int size : x)
            System.out.print(size);
    }
}
```

O resultado da execução do código precedente, desenvolvido em Java, será 6.

**Comentários**:
Pessoal, esse for (int size : x) é um laço que percorre cada valor do array e o imprime um ao lado do outro (note o uso de `print` em vez de `println`). O array tem os valores 1, 3, 7, 22 e 51. O código vai imprimir 1372251 na tela, e não o número 6 como afirma a questão.

**Gabarito**: E

**30. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: FUB Prova: CESPE / CEBRASPE - 2025 - FUB - Técnico de Tecnologia da Informação)**
No que se refere à linguagem de programação Java, julgue o item subsecutivo.

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
```

```
                System.out.println("Estudante " + i + ", Prova " + j);
            }
        }
    }
}
```

**Comentários**:
Temos um laço externo que vai de 1 a 3 (3 voltas) e, para cada uma dessas voltas, um laço interno que vai de 1 a 2 (2 voltas). Multiplicando as iterações (3×2), teremos exatamente 6 execuções do comando System.out.println.

**Gabarito**: C

**31. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: EMBRAPA Prova: CESPE / CEBRASPE - 2025 - EMBRAPA - Analista – Área: Gestão da Informação – Subárea: Engenharia de Software)**

Julgue o próximo item, a respeito de arquitetura de sistemas de computação, sistemas de informação e linguagens de programação.
Em Java, a palavra-chave static é utilizada para declarar variáveis locais dentro de um método, para que elas mantenham seu valor entre diferentes chamadas ao método.

**Comentários**:
A afirmação sobre static está errada: em Java, você não pode usar static em variáveis locais (dentro de métodos); ele é usado para membros da classe como um todo.

**Gabarito**: E

**32. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Técnico Judiciário – Área: Apoio Especializado – Especialidade: Programação de Sistemas)**

Julgue o próximo item, relativos a linguagens e tecnologias de programação.

Considere o código a seguir, escrito em Java.

```
class Veiculo {
    public void x() {
        System.out.println("movimenta");
    }
}

class Aviao extends Veiculo {
    public void x() {
        System.out.println("voa");
    }
}

class Main {
    public static void main(String[] args) {
        Veiculo mVeiculo = new Veiculo();
        Veiculo mAviao = new Aviao();
```

```
        mVeiculo.x();
        mAviao.x();
    }
}
```

No código precedente há um exemplo de polimorfismo; ao ser executado, o código apresentará o seguinte resultado:

```
movimenta

voa
```

**Comentários**:
Perfeito! Como mAviao é um objeto do tipo Aviao (mesmo em uma variável de referência Veiculo), o método chamado é o da subclasse, imprimindo "voa".

**Gabarito**: C

**33. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Técnico Judiciário – Área: Apoio Especializado – Especialidade: Programação de Sistemas)**

Julgue o próximo item, relativos a linguagens e tecnologias de programação.

Considere o seguinte código, escrito em Java.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> orgao = new ArrayList<String>();
        orgao.add("TRE-PR");
        orgao.add("TRE-BA");
        orgao.add("TRE-PI");
        orgao.add("TRE-RS");
        orgao.remove(1);
        orgao.set(1, "TRE-SC");
        System.out.println(orgao);
    }
}
```

Esse código, ao ser executado, apresentará o resultado a seguir.

```
[TRE-PR, TRE-SC, TRE-RS]
```

**Comentários**:
Vamos seguir os passos da lista:

- Lista inicial: [TRE-PR, TRE-BA, TRE-PI, TRE-RS]

- orgao.remove(1): Remove quem está no índice 1 (TRE-BA). Lista vira: [TRE-PR, TRE-PI, TRE-RS]
- orgao.set(1, "TRE-SC"): Substitui quem está no novo índice 1 (TRE-PI) por "TRE-SC".
- Lista final: [TRE-PR, TRE-SC, TRE-RS] O resultado bate exatamente com o que a banca afirmou.

**Gabarito**: C

**34. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Analista Judiciário – Área: Apoio Especializado – Especialidade: Tecnologia da Informação)**

No que concerne a Java, Zuul e Spring Cloud, julgue o item a seguir.
No código a seguir, desenvolvido em Java, a substituição dos caracteres #XPTO por @extends fará que a execução do código apresente BMW x3 como resultado.

```
public class Veiculo {
    protected String nome;
    public Veiculo(String nome) {
        this.nome = "BMW";
    }
    public void Modelo() {
    }
    public static void main(String[] args) {
        Veiculo Veiculo = new Veiculo("");
        Veiculo.Modelo();
        Veiculo bmw = new Veiculo("") {
            #XPTO
            public void Modelo() {
                System.out.println(nome + " " + "x3");
            }
        };
        bmw.Modelo();
    }
}
```

**Comentários**:
Pessoal, cuidado! A sintaxe @extends não existe no Java, resultando em erro de compilação. Se #XPTO fosse simplesmente removido, o código funcionaria!

**Gabarito**: E

**35. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: LNA Prova: CESPE / CEBRASPE - 2024 - LNA - Tecnologista – Especialidade: Desenvolvimento e Arquitetura de Software, adaptado**

```
public class Main {
    public static void main(String[] args) {
        int x = 0;
        System.out.println("Resultado");

        while (x < 7) {
            if (x % 2 != 1) {
```

```
                ++x;
                continue;
            }
            System.out.println(x);
            ++x;
        }
    }
}
```

Considerando que o código precedente tenha sido desenvolvido em Java, o resultado exibido será

```
Resultado

1

3

5
```

**Comentários**:
Gente, vamos rastrear esse código com calma. O segredo está na condição do if: x % 2 != 1. Isso é o mesmo que dizer "se o número for par".

- Começamos com x=0 (par): incrementa para 1 e o continue pula para a próxima volta.
- Agora x=1 (ímpar): o if é falso, então ele imprime 1 e incrementa para 2.
- x=2 (par): incrementa para 3 e pula.
- x=3 (ímpar): imprime 3 e incrementa para 4.
- x=4 (par): incrementa para 5 e pula.
- x=5 (ímpar): imprime 5 e incrementa para 6.
- x=6 (par): incrementa para 7 e pula.
- x=7: o laço termina. O resultado impresso é exatamente 1, 3 e 5, como a questão afirmou.

**Gabarito**: C

**36. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAGEPA - PB Prova: CESPE / CEBRASPE - 2024 - CAGEPA - PB - Analista de Sistemas - Sistemas de TI)**

```
public class Cagepa {
    public static void main(String[] args) {
        Cagepa ip = new Cagepa();
        System.out.println("Resultado=" +
        ip.Cagepa(5));
    }

    private int Cagepa(int x) {
        if(x == 0)
            return 0;
        if(x % 2 != 0)
```

```
            return x + Cagepa(x - 1) + 4;
        return Cagepa(x - 1);
    }
}
```

Considerando o código precedente, desenvolvido em Java, assinale a opção que corresponde à correta execução desse código.
A) Resultado=12
B) Resultado=21
C) Resultado=4
D) Resultado=9
E) Resultado=14

**Comentários**:
Galera, essa é uma questão de recursividade. Vamos destrinchar a chamada ip.Cagepa(5):

- Cagepa(5) é ímpar: retorna 5 + Cagepa(4) + 4.
- Cagepa(4) é par: apenas chama Cagepa(3).
- Cagepa(3) é ímpar: retorna 3 + Cagepa(2) + 4.
- Cagepa(2) é par: apenas chama Cagepa(1).
- Cagepa(1) é ímpar: retorna 1 + Cagepa(0) + 4.
- Cagepa(0) retorna 0. Agora somamos tudo: 5 + (3 + (1 + 0 + 4) + 4) + 4. Fazendo a conta: 5 + 3 + 1 + 4 + 4 + 4 = 21.

**Gabarito: B**

**37. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: SEPLAG-CE Prova: CESPE / CEBRASPE - 2024 - SEPLAG-CE - Analista de Gestão Pública - Área de Atuação: Ciência da Computação)**
Considerando linguagens de programação, julgue o item que se segue.
O código Java a seguir, após executado, apresentará 140 como resultado.

```
public class Main {
    public static void main(String[] args) {
        int soma = 0;
        for (int i = 5; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                soma += j * i;
            }
        }
        System.out.println(soma);
    }
}
```

**Comentários:**
Pessoal, aqui temos dois laços for onde o valor de i dita quantas vezes o laço de j vai rodar.

- Para i=5: soma recebe 5 * (1+2+3+4+5) = 75.
- Para i=4: soma recebe 4 * (1+2+3+4) = 40.

- Para i=3: soma recebe 3 * (1+2+3) = 18.
- Para i=2: soma recebe 2 * (1+2) = 6.
- Para i=1: soma recebe 1 * (1) = 1.
- Somando os resultados (75 + 40 + 18 + 6 + 1), chegamos exatamente ao valor de 140.

**Gabarito**: C

**38. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-GO Prova: CESPE / CEBRASPE - 2024 - MPE-GO - Analista em Informática)**

Julgue o próximo item, relativo a Java, JavaScript e Python.

```
class Main {
    public static void main(String[] args) {

        char[] letras = {'M', 'P', 'G', 'O'};

        for (char x: letras) {
            System.out.println(x);
        }
    }
}
```

Considerando-se que o código precedente foi desenvolvido em Java, é correto afirmar que o resultado, após sua execução, será:

```
0

1

2

3
```

**Comentários**:

Gente, o laço for (char x: letras) é um "enhanced for" que percorre cada caractere do array e o atribui à variável x. O array contém 'M', 'P', 'G' e 'O'.

Portanto, o comando System.out.println(x) vai imprimir cada uma dessas letras em uma linha diferente. A afirmação de que o resultado seria a sequência numérica 0, 1, 2, 3 está completamente errada, pois o código imprime os valores e não os índices.

**Gabarito**: E

**39. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: STJ Prova: CESPE / CEBRASPE - 2024 - STJ - Analista Judiciário - Área: Apoio Especializado - Especialidade: Suporte em Tecnologia da Informação)**

Considere o seguinte código, escrito em Java.

```
class Main {
    public static void main(String[] args) {
```

```
        int x = 11, y = -7, z = 22;
        int resultado = (x >= y) ? ((x >= z) ? x : z) : ((y >= z) ? y : z);

        System.out.println("Resultado: " + resultado);
    }
}
```

A partir do código precedente, é correto afirmar que a execução desse código apresentará o seguinte resultado.

```
Resultado: 22
```

**Comentários**:
Galera, o operador ternário funciona como um if-else compacto. A expressão é: (condição) ? (se verdadeiro) : (se falso). Como x=11, y=-7 e z=22:

- A primeira condição (x >= y) é (11 >= -7), que é VERDADEIRO.
- Entramos na primeira parte: ((x >= z) ? x : z).
- Avaliamos (11 >= 22), que é FALSO.
- O resultado escolhido é z, que vale 22. Portanto, o valor final de resultado será 22.

**Gabarito**: C

**40. (Ano: 2024 Banca: CESGRANRIO Órgão: IPEA Prova: CESGRANRIO - 2024 - IPEA - Técnico de Planejamento e Pesquisa - Desenvolvimento de Sistemas)**

Sejam as seguintes classes Java, que ocupam, cada qual, o seu próprio arquivo:

```
package excecao;
public class XP01 extends Exception {
    public XP01() {
    }
}
package excecao;
public class XP1A extends XP01 {
    public XP1A() {
    }
}
package excecao;
public class XP02 extends Exception {
    public XP02() {
    }
}
package excecao;
public class XP2A extends XP02 {
    public XP2A() {
    }
}
```

```
import excecao.*;
public class IPx {
    public void fnc01(Object o) throws XP01, XP2A {
    }
}
```

Qual classe poderá ser inserida em um projeto, juntamente com as classes acima, sem produzir erros de compilação?

(A)
```
import excecao.*;
public class IPa extends IPx {
    public void fnc01(Object o) throws Exception, Exception {
    }
}
```

(B)
```
import excecao.*;
public class IPb extends IPx {
    public void fnc01(Object o) throws XP2A, XP1A {
    }
}
```

(C)
```
import excecao.*;
public class IPc extends IPx {
     public void fnc01(Object o) throws XP02 {
     }
}
```

(D)
```
import excecao.*;
public class IPd extends IPx {
    public void fnc01(Object o) throws XP01, XP02 {
    }
}
```

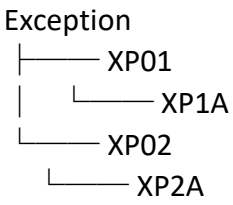
(E)
```
import excecao.*;
public class IPe extends IPx {
    public void fnc01(Object o) throws XP01, XP02 {
    }
}
```

**Comentários:**

Quando sobrescrevemos um método (`override`) em Java, o método na subclasse não pode lançar exceções mais genéricas do que as declaradas no método da classe-pai. A regra detalhada diz que, ao sobrescrever:

- Pode lançar as mesmas exceções declaradas na superclasse.
- Pode lançar exceções mais específicas (subclasses) das declaradas na superclasse.
- Pode lançar menos exceções ou nenhuma exceção.
- Não pode lançar exceções mais amplas (genéricas ou diferentes) do que as definidas no método original da superclasse.

No nosso código, temos a seguinte hierarquia:




E a superclasse fornecida:

```
import excecao.*;

public class IPx {
   public void fnc01(Object o) throws XP01, XP2A {
   }
}
```

Portanto, qualquer classe que sobrescreva o método `fnc01` poderá lançar somente:

- As mesmas exceções: XP01 e/ou XP2A
- Exceções mais específicas (ex: XP1A)
- Nenhuma exceção

Mas nunca exceções diferentes ou mais amplas (ex: XP02 ou Exception).

Com isso em mente, vamos analisar as alternativas.

a) Errado. Aqui temos um erro - Exception é mais genérico que XP01 e XP2A.
b) Certo. Temos XP2A igual à superclasse, e XP1A mais específico, em relação à XP01, portanto temos uma implementação correta.
c) Errado. XP02 não está relacionada a nenhuma exceção declarada na superclasse (superclasse declara XP2A, que é filha de XP02, mas não é possível subir na hierarquia).
d) Errado. XP01 é igual ao método original, sendo válido, mas XP02 não foi declarada no método original e é mais genérica que XP2A (proibido).
e) Errado. Apresenta o mesmo erro que a letra D.

Portanto, correta a letra B.

**Gabarito:** Letra B

**41. (Ano: 2024 Banca: CESGRANRIO Órgão: Caixa Prova: CESGRANRIO - 2024 - Caixa - Técnico Bancário Novo - Tecnologia da Informação)**

Seja a seguinte classe Java, que ocupa o seu próprio arquivo:

```
public abstract class Klm {
        void lm01() {
        }
        protected final void imprime() {
        }
        protected abstract void lm02();
}
```

Qual classe poderá ser inserida em um projeto, juntamente com a classe acima, sem produzir erros de compilação?

(A)

```
public class Cla extends Klm {
        private void lm01() {
        }
        public void lm02() {
        }
    }
```

(B)

```
public abstract class Clb extends Klm {
       public void imprime(String s) {
       }
    }
```

(C)

```
public abstract class Clc extends Klm {
        void lm03() {
       }
       public void imprime() {
       }
   }
```

(D)

```
public class Cld extends Klm {
        void fg01() {
        }
        public void imprime(String s) {
        }
   }
```



02648216103 - HENRIQUE AMORIM BARRETO

(E)

```
public class Cle extends Klm {
        void fg02() {
        }
        public void imprime() {
        }
        public void lm02() {
        }
    }
```

**Comentários:**

A CESGRANRIO é campeã em fazer questões de nível muito avançado em Java. Na nossa classe base fornecida:

```
public abstract class Klm {
   void lm01() {
   }

   protected final void imprime() {
   }

   protected abstract void lm02();
}
```

Temos a seguinte situação:

- lm01() é default (package-private) → pode ser sobrescrito, mas deve manter ou aumentar a visibilidade.
- imprime() é protected final → não pode ser sobrescrito.
- lm02() é protected abstract → deve obrigatoriamente ser implementado em subclasses não abstratas.

Com esse panorama geral, conseguimos analisar as alternativas.

a) Errado. O método lm01() da superclasse tem visibilidade default (package-private). Ao sobrescrevê-lo com private, a visibilidade está sendo reduzida, o que viola a regra da sobrescrita.
b) Certo. A classe Clb é abstrata, então não precisa implementar lm02() ainda. Também, o método imprime(String s) não sobrescreve imprime(), pois tem uma assinatura diferente (parâmetros), portanto não viola a cláusula final.
c) Errado. O método imprime() da superclasse é final. Aqui, está sendo sobrescrito, o que não é permitido. Mesmo sendo uma classe abstrata, sobrescrever método final gera erro.
d) Errado. Cld não é abstrata, mas não implementa o método abstrato lm02() da superclasse, o que gera erro de compilação. Já o método imprime(String s) é apenas um novo método, não sobrescrevendo imprime() da superclasse – sendo uma ação válida.

e) Errado. O método imprime() está sendo sobrescrito, mas na superclasse ele é final, isso não é permitido. Mesmo com lm02() corretamente implementado, a violação da cláusula final causa erro.

Portanto, correta a letra B.

**Gabarito:** Letra B

---

**42. (Ano: 2024 Banca: FGV Órgão: TJ-MS Prova: FGV - 2024 - TJ-MS - Técnico de Nível Superior - Analista de Sistemas Computacionais - Web Designer)**

Considere o seguinte código em Java:

```
public static void main(String[] args)
  throws MalformedURLException {
  URL url = new URL("https://bloco13:trib@"+
     "www.tjms.jus.br/ws?id=0#A");

String s1 = url.getRef(), s2 = url.getUserInfo(),
  s3 = url.getQuery(), s4 = url.getFile();

Arrays.asList(s1,s2,s3,s4).stream()
  .map(s->s.concat(";"))
  .forEach(System.out::print);
}
```

Ao ser executado com as importações necessárias, o código acima imprime na saída padrão:

a) Exception;
b) A;bloco13;id=0;null;
c) null;bloco13:trib;id=0#A;null;
d) A;bloco13:trib;id=0;/ws?id=0;
e) null;bloco13;id=0#A;/ws?id=0#A;

**Comentários:**

A URL fornecida é a seguinte: https://bloco13:trib@www.tjms.jus.br/ws?id=0#A. Temos 4 strings (s1, s2, s3, s4) criadas a partir dessa URL, com métodos diferentes:

| Método | Retorno |
|---|---|
| getRef() | "A" (fragmento após #) |
| getUserInfo() | "bloco13:trib" (antes do @) |
| getQuery() | "id=0" |
| getFile() | "/ws?id=0" (path + query) |

A array criada mapeia essas 4 strings e faz um print com cada elemento, em ordem. Ou seja, a saída será:

A;bloco13:trib;id=0;/ws?id=0;



02648216103 - HENRIQUE AMORIM BARRETO

Portanto, correta a letra D

**Gabarito:** Letra D

## Funcionalidades recentes

**43. (Ano: 2024 Banca: FGV Órgão: CGE-PB Prova: FGV - 2024 - CGE-PB - Auditor de Contas Públicas - Auditoria de Tecnologia da Informação)**

O desenvolvedor José implementou a classe CGEPrincipal em Java 17. A fim de tornar a implementação mais aderente ao modelo de domínio, José restringiu a herança da classe na declaração, especificando que a CGEPrincipal poderia ser herdada apenas pelas classes CGEUm e CGEDois.
Para restringir a herança na declaração da classe CGEPrincipal, José utilizou a funcionalidade do Java 17:
A) reflection;
B) static class;
C) sealed class;
D) foreign function;
E) pattern matching.

**Comentários:**

O conceito chave do enunciado é restringir a herança. Lembrem da nossa analogia da festa VIP. No Java antigo, ou todo mundo entrava na festa estendendo a classe de forma livre, ou ninguém entrava porque a classe tinha a palavra final. O desenvolvedor José precisava de um meio-termo: ele queria que apenas as classes CGEUm e CGEDois entrassem.

A única funcionalidade do Java que cria essa lista VIP na declaração da classe é a **Sealed Class**, ou classe selada.

Vamos passar rapidamente pelas outras opções para você não ter dúvidas. A opção reflection serve para inspecionar o código em tempo de execução, lendo métodos e atributos dinamicamente, não servindo para restringir herança. A opção static class refere-se a classes aninhadas que não dependem da instância da classe externa. A opção foreign function é um recurso para chamar códigos escritos em C ou C++ fora da máquina virtual. Por fim, pattern matching é um recurso para checar o tipo e converter variáveis em uma única tacada.

**Gabarito: C**

**44. (Ano: 2025 Banca: FGV Órgão: MPU Prova: FGV - 2025 - MPU - Analista do MPU - Desenvolvimento de Sistemas)**

Considere o código em Java a seguir.

```
package com.mpu.questao;

public sealed class MPU {}
final class MPF extends MPU {}
```

```
sealed class MPM extends MPU {}
final class MPDFT extends MPU {}
non-sealed class MPT extends MPU {}
```

O código acima possui um erro, pois a classe:

A) MPU, sendo sealed, omitiu a cláusula permits;
B) MPF, sendo final, não pode estender a classe MPU, que é sealed;
C) MPM, sendo sealed, não definiu nenhuma subclasse permitida;
D) MPT, sendo non-sealed, não pode estender a classe MPU, que é sealed;
E) MPDFT, sendo final, não pode ser declarada após a classe MPM, de escopo protected.

**Comentários:**

Gente, prestem muita atenção aqui. Essa é a pegadinha de ouro da FGV. Eles subiram o nível e foram cobrar regras estritas de compilação.

O código apresenta uma classe selada principal chamada MPU. Percebam que ela **não tem a palavra permits**. A alternativa A diz que isso é um erro. Mas atenção: no Java, se as subclasses estiverem declaradas exatamente no mesmo arquivo de código-fonte, a cláusula permits pode ser omitida. O compilador é inteligente o suficiente para ler o arquivo e fechar a lista VIP sozinho. Logo, a alternativa A é falsa, o erro não está ali.

O erro real está na alternativa C. O desenvolvedor criou uma classe chamada MPM e colocou a palavra sealed nela. Mas olhem o código: **não existe nenhuma outra classe no arquivo que estenda a classe MPM**.

A regra da máquina virtual Java é absoluta: **toda classe selada nasceu para ser herdada**. Colocar um segurança na porta de uma área VIP que não tem nenhum convidado gera um erro de compilação imediato. Uma classe selada sem subclasses é um paradoxo no Java e não compila.

**Gabarito: C**

**45. (Ano: 2025 Banca: CONSULPAM Órgão: CONAB Prova: CONSULPAM - 2025 - CONAB - Analista - Tecnologia da Informação (Desenvolvimento))**

Considere um profissional que está utilizando o Java, Java Development Kit (JDK), 17, para desenvolver uma aplicação envolvendo formas geométricas. Em determinado momento, ele constrói o seguinte código:

```
sealed interface Forma permits Circulo, Retangulo
{}

final class Circulo implements Forma {
    double raio;
    Circulo(double r) { this.raio = r; }
}

final class Retangulo implements Forma {
    double largura, altura;
    Retangulo(double l, double a) { this.largura = l; this.altura = a; }
```

```
}

public class Main {
    public static void main(String[] args) {
        Forma f = new Circulo(2.0);
        String tipo = switch (f) {
            case Circulo c -> "É um círculo com raio " + c.raio;
            case Retangulo r -> "É um retângulo de " + r.largura + "x" + r.altura;
        };
        System.out.println(tipo);
    }
}
```

Acerca do cenário apresentado, avalie as sentenças a seguir:

A instrução switch com pattern matching é uma funcionalidade estável no JDK 17, não exigindo ativação com --enable-preview. PORQUE O uso de sealed interface permite restringir a herança a tipos específicos, e combinado com switch exaustivo, permite ao compilador garantir que todos os casos foram tratados.
Acerca dessas sentenças, assinale a alternativa CORRETA.

A) As duas sentenças são verdadeiras, e a segunda é uma justificativa correta da primeira.
B) As duas sentenças são verdadeiras, mas a segunda não é uma justificativa correta da primeira.
C) A primeira sentença é verdadeira, e a segunda é falsa.
D) A primeira sentença é falsa, e a segunda é verdadeira.

**Comentários:**
Vamos quebrar o problema ao meio para facilitar.

A primeira sentença diz que a instrução switch com pattern matching é uma funcionalidade estável no JDK 17, não exigindo ativação com enable-preview. Isso é falso. As Sealed Classes se tornaram estáveis e oficiais no Java 17. Porém, o Pattern Matching aplicado ao Switch entrou no Java 17 apenas como uma versão de testes, o que chamamos de preview. Para rodar isso no Java 17, você obrigatoriamente precisava passar um parâmetro de terminal para habilitar esse preview. Essa funcionalidade **só virou estável e definitiva na versão Java 21**.

A segunda sentença afirma que o uso de sealed interface permite restringir a herança a tipos específicos, e combinado com switch exaustivo, permite ao compilador garantir que todos os casos foram tratados. Isso é verdadeiro. Esse é o benefício máximo de juntar as duas tecnologias. Como a interface Forma é selada e só permite Circulo e Retangulo, o compilador sabe que não existe nenhuma outra forma geométrica possível no ecossistema do sistema. Quando você monta o switch cobrindo o Circulo e o Retangulo, o compilador atesta que **a checagem foi exaustiva**, ou seja, esgotou todas as possibilidades. É por isso que o código compila perfeitamente sem precisar daquela cláusula default no final do switch.

Como a primeira sentença é falsa e a segunda é verdadeira, a alternativa que reflete nossa análise é a letra D.

**Gabarito: D**

# LISTA DE QUESTÕES

## Teoria do Java

1. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática)**

Com relação às linguagens de programação Java e PHP, julgue o item a seguir.
Na linguagem Java, os tipos int, char e long são exemplos de dados elementares, e os tipos matrizes e vetores são dados estruturados da linguagem.

2. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática), adaptada**

Julgue o próximo item, relativo a controle de dados, compilação e engenharia de software.
Os programas em Java podem ser tanto interpretados quanto compilados.

3. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Analista Judiciário - Área Apoio Especializado - Especialidade: Analista de Sistemas)**

Acerca das tecnologias que atendem a PDPJ-Br, julgue o próximo item.

Em Java, uma subclasse herda os membros, campos e métodos de sua superclasse direta, podendo acrescentar novos membros, desde que mantenha intactos os membros herdados.

4. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Analista Judiciário - Área Apoio Especializado - Especialidade: Analista de Sistemas)**

Acerca das tecnologias que atendem a PDPJ-Br, julgue o próximo item.

As interfaces do Java permitem que objetos de classes não relacionadas sejam processados de forma polimórfica.

5. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Técnico Judiciário - Área Apoio Especializado - Especialidade: Programação de Sistemas)**

Julgue o item seguinte, relativos a Java.
Em Java, as classes possuem herança múltipla, de modo que cada classe pode ser derivada de mais de uma superclasse direta.

6. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Técnico Judiciário - Área Apoio Especializado - Especialidade: Programação de Sistemas)**

Julgue o item seguinte, relativos a Java.
O polimorfismo em Java permite processar objetos que derivam da mesma superclasse, direta ou indiretamente; cada objeto pode executar ações diferentes a partir da mesma chamada.

7. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Analista de Sistemas)**

A respeito de linguagens de programação, julgue o próximo item.

A linguagem Java suporta arrays multidimensionais de forma direta, sem a necessidade de manipulação em cascata de arrays unidimensionais ou bidimensionais.

8. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Analista de Sistemas)**

A respeito de linguagens de programação, julgue o próximo item.
Os bytecodes gerados pelo compilador Java independem da plataforma em que são executados, diferentemente das instruções em linguagem de máquina.

9. **(Ano: 2025 Banca: CESPE / CEBRASPE Órgão: Polícia Federal Prova: CESPE / CEBRASPE - 2025 - Polícia Federal - Perito Criminal Federal - Área 3: Informática Forense)**

Julgue os próximos itens, a respeito das características e das aplicações básicas das linguagens Java e JavaScript.
Devido à compilação do código-fonte em bytecode, que é executado pela máquina virtual Java (JVM), os programas Java podem ser executados em diversos sistemas operacionais, sem a necessidade de recompilação.

10. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: Prefeitura de Cachoeiro de Itapemirim - ES Prova: CESPE / CEBRASPE - 2024 - Prefeitura de Cachoeiro de Itapemirim - ES - Técnico em Informática)**

Julgue o próximo item, relativo a controle de dados, compilação e engenharia de software.
Na linguagem Java, existe um mecanismo próprio de gerenciamento de memória denominado garbage collector.

11. **(Ano: 2025 Banca: CESPE / CEBRASPE Órgão: TRT - 10ª REGIÃO (DF e TO) Prova: CESPE / CEBRASPE - 2025 - TRT - 10ª REGIÃO (DF e TO) - Analista Judiciário – Área: Apoio Especializado – Especialidade: Tecnologia da Informação)**

Acerca de desenvolvimento de sistemas, julgue o próximo item.
Em Java, uma das maneiras de se implementar o polimorfismo é a partir de interfaces que permitem o tratamento de objetos de classes diferentes de forma uniforme.

12. **(Ano: 2025 Banca: CESPE / CEBRASPE Órgão: SUSEP Prova: CESPE / CEBRASPE - 2025 - SUSEP - Analista Técnico: - Área: Tecnologia da Informação e Ciência de Dados)**

Julgue o item que se segue, referente a HTML, CSS e JavaScript, Python e Java.
No Java, versão 21, o bloqueio de uma virtual thread, por exemplo, em uma operação de I/O, bloqueia a platform thread subjacente.

13. **(Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-GO Prova: CESPE / CEBRASPE - 2024 - MPE-GO - Analista em Informática)**

Julgue o item seguinte, com base nos conceitos de linguagem de programação Ruby e Java.
Ao se utilizar a estrutura try-catch-finally para tratar certos tipos de erros em um programa Java, as instruções do bloco finally são úteis para realizar operações, independentemente do resultado do bloco try.

14. **(Ano: 2025 Banca: IMPARH Órgão: CGM de Fortaleza - CE Prova: IMPARH - 2025 - CGM de Fortaleza - CE - Auditor de Controle Interno - Área 2 (Ciências da Computação))**

Sobre os conceitos relacionados à orientação a objetos na linguagem de programação Java, marque a opção correta.
A) Java permite herança múltipla de classes diretamente.

B) Todos os métodos de uma classe abstrata devem ser abstratos.
C) Classes em Java podem implementar várias interfaces.
D) Uma classe com métodos e variáveis públicas não pode ser encapsulada.

**15. (Ano: 2025 Banca: FGV Órgão: PC-MG Prova: FGV - 2025 - PC-MG - Perito Criminal - Área II)**

Considere as características fundamentais da linguagem Java e seus recursos no desenvolvimento de aplicações.
Com base nos conceitos básicos de orientação a objetos, em relação ao uso de classes e objetos no Java, assinale a afirmativa correta.
A) Em Java, uma classe pode ser instanciada apenas se tiver um construtor público ou um construtor com especificadores de acesso adequados.
B) O modificador public impede que um método ou variável de instância seja acessado fora da sua própria classe.
C) Uma classe semi-abstrata não pode ser instanciada diretamente, mas pode conter métodos implementados e não implementados.
D) O conceito de polimorfismo permite que um objeto se comporte de forma diferente dependendo do tipo da referência usada para acessá-lo.
E) O uso da palavra-chave super é utilizado para acessar membros da classe pai em classes que herdam de outra classe.

**16. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.

O Java oferece uma forte tipagem estática, o que significa que os tipos de dados de variáveis devem ser explicitamente definidos em tempo de compilação, garantindo maior segurança e detecção precoce de erros.

**17. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

As linguagens de programação são responsáveis por tirar do papel os algoritmos criados pelo programador e por transformá-los em programas para serem executados pelos computadores modernos. Considerando essa informação, julgue o item seguinte.
Em Java, as variáveis de tipo primitivo não consomem memória, pois são apenas referências a objetos armazenados na heap.

**18. (Ano: 2025 Banca: Quadrix Órgão: CFO Prova: Quadrix - 2025 - CFO - Técnico em Tecnologia da Informação)**

Em Java, os tipos de dados primitivos são passados por referência, permitindo que suas mudanças dentro de um método afetem diretamente a variável original.

**19. (Ano: 2024 Banca: IBFC Órgão: TRF - 5ª REGIÃO Prova: IBFC - 2024 - TRF - 5ª REGIÃO - Técnico Judiciário - Área de Apoio Especializado - Especialidade Desenvolvimento de Sistemas da Informação)**

A linguagem Java é amplamente utilizada mundialmente, devido a sua robustez e consolidação na área da computação. Diante do exposto, analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).

( ) A Máquina Virtual Java (JVM) permite que os programas Java sejam executados em diferentes plataformas, proporcionando portabilidade.
( ) Java não suporta herança, um dos princípios fundamentais da programação orientada a objetos.
( ) Java suporta coleta de lixo, o que significa que o programador não precisa gerenciar explicitamente a alocação e desalocação de memória.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

a) V - V - V
b) V - F - V
c) V - F - F
d) F - V - V

**20. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: FINEP Prova: CESPE / CEBRASPE - 2024 - FINEP - Analista - Tecnologia da Informação - Software)**

Assinale a opção em que é apresentado o principal método usado na linguagem Java para iniciar a execução de um programa.

a) execute( )
b) run( )
c) start( )
d) begin( )
e) main( )

**21. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: FINEP Prova: CESPE / CEBRASPE - 2024 - FINEP - Analista - Tecnologia da Informação - Software)**

Assinale a opção em que é indicado o modificador de acesso em Java que pode permitir que os membros de uma classe sejam acessados apenas por classes do mesmo pacote.

a) package-private
b) internal
c) static
d) public
e) private

**22. (Ano: 2024 Banca: FUNCERN Órgão: IF-RN Prova: FUNCERN - 2024 - IF-RN - Técnico de Tecnologia da Informação)**

Um item de dado de programa pode ser definido como uma constante ou variável. As constantes, por exemplo, possuem um valor predefinido e que não mudam durante a execução do programa. Já as variáveis são um item de dado cujo valor pode ser alterado durante a execução do programa. Nesse contexto, a palavra-chave usada na linguagem Java para declarar uma constante é

a) const.
b) final.

c) let.
d) protected.

**23. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

Na linguagem Java, para impedir que um método criado em uma classe J não possa ser sobrescrito por alguma outra classe K que herde de J, deve-se usar a palavra

a) extends
b) final
c) implements
d) super
e) void

**24. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

Em um projeto desenvolvido na linguagem Java, considere um método P que chama um método Q. Existe uma palavra que, quando colocada na assinatura do método Q, faz com que, caso ocorra uma exceção nesse método, o tratamento dessa exceção seja da responsabilidade do método P.

Qual é essa palavra?

a) catch
b) Exception
c) finally
d) throws
e) try

**25. (Ano: 2024 Banca: CESGRANRIO Órgão: UNEMAT Prova: CESGRANRIO - 2024 - UNEMAT - Analista de Sistemas)**

É uma característica da linguagem Java conseguir representar a

a) codificação de métodos completos em interfaces
b) herança múltipla de interface
c) herança múltipla de classe
d) instanciação de classes abstratas
e) instanciação de interfaces

# Códigos em Java

**26. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-TO Prova: CESPE / CEBRASPE - 2024 - MPE-TO - Analista Ministerial Especializado - Área de Atuação: Análise de Sistemas)**

Em relação à linguagem de programação Java, julgue o item a seguir.
Durante a execução do código a seguir, ocorrerá uma exceção ArithmeticException.

```java
public class Main {
    public static void main(String[] args) {
        int numero = 10;
        int divisor = 0;

        try {
            int resultado = numero / divisor;
            System.out.println("Resultado: " + resultado);
        } catch (ArithmeticException e) {
            e.printStackTrace();
        }
    }
}
```

**27. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: EMBRAPA Prova: CESPE / CEBRASPE - 2025 - EMBRAPA - Analista – Área: Gestão da Informação – Subárea: Engenharia de Infraestrutura e Tecnologia da Informação)**

Julgue o seguinte item, referente à programação e estruturas de dados.

O código a seguir, em Java, retornará `Tecnológica` como resultado de sua execução.

```java
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> embrapa = new ArrayList<String>();

        embrapa.add("Ageitec");
        embrapa.add("Agência");
        embrapa.add("Embrapa");
        embrapa.add("de Informação");
        embrapa.add("Tecnológica");

        int a = 1;

        for (int i = 0; i < embrapa.size() - 2; ++i) {
            a = a + 2;
        }
        System.out.println(embrapa.get(a - 3));
    }
```

```
}
```

**28. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: FUB Prova: CESPE / CEBRASPE - 2025 - FUB - Técnico de Tecnologia da Informação)**

No que se refere à linguagem de programação Java, julgue o item subsecutivo.

Considere a seguinte classe **Estudante**:

```
public class Estudante {
    private String nome;
    private int matricula;

    public Estudante(String nome, int matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }
}
```

Considere, também, o seguinte trecho de código, que visa cadastrar e exibir os dados de um estudante:

```
public class Main {
    public static void main(String[] args) {
        Estudante e = new Estudante("Carlos", 12345);

        System.out.println("Nome: " + e.nome);
        System.out.println("Matrícula: " + e.matricula);
    }
}
```

A execução dos trechos de código precedentes criará um objeto da classe *Estudante* com os dados informados e exibirá as informações na tela sem apresentar erro de compilação.

**29. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: TRF - 6ª REGIÃO Prova: CESPE / CEBRASPE - 2025 - TRF - 6ª REGIÃO - Analista Judiciário – Área: Apoio Especializado – Especialidade: Análise de Dados)**

Julgue o próximo item, relativo a Java.

```
import java.io.*;
class Easy {
    public static void main(String[] args){
        int x[] = {1, 3, 7, 22, 51};
        for (int size : x)
            System.out.print(size);
    }
}
```

O resultado da execução do código precedente, desenvolvido em Java, será 6.

**30. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: FUB Prova: CESPE / CEBRASPE - 2025 - FUB - Técnico de Tecnologia da Informação)**

No que se refere à linguagem de programação Java, julgue o item subsecutivo.

A execução do código a seguir será bem-sucedida e imprimirá, em tela, 6 linhas.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("Estudante " + i + ", Prova " + j);
            }
        }
    }
}
```

**31. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: EMBRAPA Prova: CESPE / CEBRASPE - 2025 - EMBRAPA - Analista – Área: Gestão da Informação – Subárea: Engenharia de Software)**

Julgue o próximo item, a respeito de arquitetura de sistemas de computação, sistemas de informação e linguagens de programação.

Em Java, a palavra-chave static é utilizada para declarar variáveis locais dentro de um método, para que elas mantenham seu valor entre diferentes chamadas ao método.

**32. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Técnico Judiciário – Área: Apoio Especializado – Especialidade: Programação de Sistemas)**

Julgue o próximo item, relativos a linguagens e tecnologias de programação.

Considere o código a seguir, escrito em Java.

```
class Veiculo {
    public void x() {
        System.out.println("movimenta");
    }
}

class Aviao extends Veiculo {
    public void x() {
        System.out.println("voa");
    }
}

class Main {
    public static void main(String[] args) {
        Veiculo mVeiculo = new Veiculo();
        Veiculo mAviao = new Aviao();
```

```
        mVeiculo.x();
        mAviao.x();
    }
}
```

No código precedente há um exemplo de polimorfismo; ao ser executado, o código apresentará o seguinte resultado:

```
movimenta

voa
```

**33. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Técnico Judiciário – Área: Apoio Especializado – Especialidade: Programação de Sistemas)**

Julgue o próximo item, relativos a linguagens e tecnologias de programação.

Considere o seguinte código, escrito em Java.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> orgao = new ArrayList<String>();
        orgao.add("TRE-PR");
        orgao.add("TRE-BA");
        orgao.add("TRE-PI");
        orgao.add("TRE-RS");
        orgao.remove(1);
        orgao.set(1, "TRE-SC");
        System.out.println(orgao);
    }
}
```

Esse código, ao ser executado, apresentará o resultado a seguir.

```
[TRE-PR, TRE-SC, TRE-RS]
```

**34. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: TSE Prova: CESPE / CEBRASPE - 2024 - TSE - Analista Judiciário – Área: Apoio Especializado – Especialidade: Tecnologia da Informação)**

No que concerne a Java, Zuul e Spring Cloud, julgue o item a seguir.

No código a seguir, desenvolvido em Java, a substituição dos caracteres #XPTO por @extends fará que a execução do código apresente BMW x3 como resultado.

```
public class Veiculo {
    protected String nome;
```

```
    public Veiculo(String nome) {
        this.nome = "BMW";
    }
    public void Modelo() {
    }
    public static void main(String[] args) {
        Veiculo Veiculo = new Veiculo("");
        Veiculo.Modelo();
        Veiculo bmw = new Veiculo("") {
            #XPTO
            public void Modelo() {
                System.out.println(nome + " " + "x3");
            }
        };
        bmw.Modelo();
    }
}
```

**35. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: LNA Prova: CESPE / CEBRASPE - 2024 - LNA - Tecnologista – Especialidade: Desenvolvimento e Arquitetura de Software, adaptado**

```
public class Main {
    public static void main(String[] args) {
        int x = 0;
        System.out.println("Resultado");

        while (x < 7) {
            if (x % 2 != 1) {
                ++x;
                continue;
            }
            System.out.println(x);
            ++x;
        }
    }
}
```

Considerando que o código precedente tenha sido desenvolvido em Java, o resultado exibido será

```
Resultado

1

3

5
```

**36. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAGEPA - PB Prova: CESPE / CEBRASPE - 2024 - CAGEPA - PB - Analista de Sistemas - Sistemas de TI)**

```
public class Cagepa {
    public static void main(String[] args) {
        Cagepa ip = new Cagepa();
        System.out.println("Resultado=" +
        ip.Cagepa(5));
    }

    private int Cagepa(int x) {
        if(x == 0)
            return 0;
        if(x % 2 != 0)
            return x + Cagepa(x - 1) + 4;
        return Cagepa(x - 1);
    }
}
```

Considerando o código precedente, desenvolvido em Java, assinale a opção que corresponde à correta execução desse código.
A) Resultado=12
B) Resultado=21
C) Resultado=4
D) Resultado=9
E) Resultado=14

**37. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: SEPLAG-CE Prova: CESPE / CEBRASPE - 2024 - SEPLAG-CE - Analista de Gestão Pública - Área de Atuação: Ciência da Computação)**

Considerando linguagens de programação, julgue o item que se segue.
O código Java a seguir, após executado, apresentará 140 como resultado.

```
public class Main {
    public static void main(String[] args) {
        int soma = 0;
        for (int i = 5; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                soma += j * i;
            }
        }
        System.out.println(soma);
    }
}
```

**38. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPE-GO Prova: CESPE / CEBRASPE - 2024 - MPE-GO - Analista em Informática)**

Julgue o próximo item, relativo a Java, JavaScript e Python.

```
class Main {
    public static void main(String[] args) {

        char[] letras = {'M', 'P', 'G', 'O'};

        for (char x: letras) {
            System.out.println(x);
        }
    }
}
```

Considerando-se que o código precedente foi desenvolvido em Java, é correto afirmar que o resultado, após sua execução, será:

```
0

1

2

3
```

**39. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: STJ Prova: CESPE / CEBRASPE - 2024 - STJ - Analista Judiciário - Área: Apoio Especializado - Especialidade: Suporte em Tecnologia da Informação)**

Considere o seguinte código, escrito em Java.

```
class Main {
    public static void main(String[] args) {
        int x = 11, y = -7, z = 22;
        int resultado = (x >= y) ? ((x >= z) ? x : z) : ((y >= z) ? y : z);

        System.out.println("Resultado: " + resultado);
    }
}
```

A partir do código precedente, é correto afirmar que a execução desse código apresentará o seguinte resultado.

```
Resultado: 22
```

**40. (Ano: 2024 Banca: CESGRANRIO Órgão: IPEA Prova: CESGRANRIO - 2024 - IPEA - Técnico de Planejamento e Pesquisa - Desenvolvimento de Sistemas)**

Sejam as seguintes classes Java, que ocupam, cada qual, o seu próprio arquivo:

package excecao;

```
public class XP01 extends Exception {
    public XP01() {
    }
}
package excecao;
public class XP1A extends XP01 {
    public XP1A() {
    }
}
package excecao;
public class XP02 extends Exception {
    public XP02() {
    }
}
package excecao;
public class XP2A extends XP02 {
    public XP2A() {
    }
}
import excecao.*;
public class IPx {
    public void fnc01(Object o) throws XP01, XP2A {
    }
}
```

Qual classe poderá ser inserida em um projeto, juntamente com as classes acima, sem produzir erros de compilação?

(A)

```
import excecao.*;
public class IPa extends IPx {
    public void fnc01(Object o) throws Exception, Exception {
    }
}
```

(B)

```
import excecao.*;
public class IPb extends IPx {
    public void fnc01(Object o) throws XP2A, XP1A {
    }
}
```

(C)

```
import excecao.*;
public class IPc extends IPx {
     public void fnc01(Object o) throws XP02 {
     }
```

```
}
```

(D)

```
import excecao.*;
public class IPd extends IPx {
    public void fnc01(Object o) throws XP01, XP02 {
    }
}
```

(E)

```
import excecao.*;
public class IPe extends IPx {
    public void fnc01(Object o) throws XP01, XP02 {
    }
}
```

**41. (Ano: 2024 Banca: CESGRANRIO Órgão: Caixa Prova: CESGRANRIO - 2024 - Caixa - Técnico Bancário Novo - Tecnologia da Informação)**

Seja a seguinte classe Java, que ocupa o seu próprio arquivo:

```
public abstract class Klm {
        void lm01() {
        }
        protected final void imprime() {
        }
        protected abstract void lm02();
}
```

Qual classe poderá ser inserida em um projeto, juntamente com a classe acima, sem produzir erros de compilação?

(A)

```
public class Cla extends Klm {
    private void lm01() {
    }
    public void lm02() {
    }
}
```

(B)

```
public abstract class Clb extends Klm {
    public void imprime(String s) {
    }
}
```

(C)

```
public abstract class Clc extends Klm {
        void lm03() {
       }
       public void imprime() {
       }
   }

(D)
public class Cld extends Klm {
        void fg01() {
        }
        public void imprime(String s) {
        }
   }

(E)
public class Cle extends Klm {
         void fg02() {
         }
        public void imprime() {
        }
        public void lm02() {
        }
   }
```

**42. (Ano: 2024 Banca: FGV Órgão: TJ-MS Prova: FGV - 2024 - TJ-MS - Técnico de Nível Superior - Analista de Sistemas Computacionais - Web Designer)**

Considere o seguinte código em Java:

```
public static void main(String[] args)
  throws MalformedURLException {
  URL url = new URL(“https://bloco13:trib@”+
     ”www.tjms.jus.br/ws?id=0#A");

String s1 = url.getRef(), s2 = url.getUserInfo(),
  s3 = url.getQuery(), s4 = url.getFile();

Arrays.asList(s1,s2,s3,s4).stream()
  .map(s->s.concat(";"))
  .forEach(System.out::print);
}
```

Ao ser executado com as importações necessárias, o código acima imprime na saída padrão:

a) Exception;
b) A;bloco13;id=0;null;

c) null;bloco13:trib;id=0#A;null;
d) A;bloco13:trib;id=0;/ws?id=0;
e) null;bloco13;id=0#A;/ws?id=0#A;

## Funcionalidades recentes

**43. (Ano: 2024 Banca: FGV Órgão: CGE-PB Prova: FGV - 2024 - CGE-PB - Auditor de Contas Públicas - Auditoria de Tecnologia da Informação)**

O desenvolvedor José implementou a classe CGEPrincipal em Java 17. A fim de tornar a implementação mais aderente ao modelo de domínio, José restringiu a herança da classe na declaração, especificando que a CGEPrincipal poderia ser herdada apenas pelas classes CGEUm e CGEDois.
Para restringir a herança na declaração da classe CGEPrincipal, José utilizou a funcionalidade do Java 17:
A) reflection;
B) static class;
C) sealed class;
D) foreign function;
E) pattern matching.

**44. (Ano: 2025 Banca: FGV Órgão: MPU Prova: FGV - 2025 - MPU - Analista do MPU - Desenvolvimento de Sistemas)**

Considere o código em Java a seguir.

```
package com.mpu.questao;

public sealed class MPU {}
final class MPF extends MPU {}
sealed class MPM extends MPU {}
final class MPDFT extends MPU {}
non-sealed class MPT extends MPU {}
```

O código acima possui um erro, pois a classe:

A) MPU, sendo sealed, omitiu a cláusula permits;
B) MPF, sendo final, não pode estender a classe MPU, que é sealed;
C) MPM, sendo sealed, não definiu nenhuma subclasse permitida;
D) MPT, sendo non-sealed, não pode estender a classe MPU, que é sealed;
E) MPDFT, sendo final, não pode ser declarada após a classe MPM, de escopo protected.

**45. (Ano: 2025 Banca: CONSULPAM Órgão: CONAB Prova: CONSULPAM - 2025 - CONAB - Analista - Tecnologia da Informação (Desenvolvimento))**

Considere um profissional que está utilizando o Java, Java Development Kit (JDK), 17, para desenvolver uma aplicação envolvendo formas geométricas. Em determinado momento, ele constrói o seguinte código:

```java
sealed interface Forma permits Circulo, Retangulo
{}

final class Circulo implements Forma {
    double raio;
    Circulo(double r) { this.raio = r; }
}

final class Retangulo implements Forma {
    double largura, altura;
    Retangulo(double l, double a) { this.largura = l; this.altura = a; }
}

public class Main {
    public static void main(String[] args) {
        Forma f = new Circulo(2.0);
        String tipo = switch (f) {
            case Circulo c -> "É um círculo com raio " + c.raio;
            case Retangulo r -> "É um retângulo de " + r.largura + "x" + r.altura;
        };
        System.out.println(tipo);
    }
}
```

Acerca do cenário apresentado, avalie as sentenças a seguir:

A instrução switch com pattern matching é uma funcionalidade estável no JDK 17, não exigindo ativação com --enable-preview. PORQUE O uso de sealed interface permite restringir a herança a tipos específicos, e combinado com switch exaustivo, permite ao compilador garantir que todos os casos foram tratados.
Acerca dessas sentenças, assinale a alternativa CORRETA.

A) As duas sentenças são verdadeiras, e a segunda é uma justificativa correta da primeira.
B) As duas sentenças são verdadeiras, mas a segunda não é uma justificativa correta da primeira.
C) A primeira sentença é verdadeira, e a segunda é falsa.
D) A primeira sentença é falsa, e a segunda é verdadeira.

# GABARITO

1. C
2. C
3. E
4. C
5. E
6. C
7. E
8. C
9. C
10. C
11. C
12. E
13. C
14. C
15. E
16. C
17. E
18. E
19. B
20. E
21. A
22. B
23. B
24. D
25. B
26. C
27. C
28. E
29. E
30. C
31. E
32. C
33. C
34. E
35. C
36. B
37. C
38. E
39. C
40. B
41. B
42. D
43. C
44. C
45. D

# ESSA LEI TODO MUNDO CONHECE:
# PIRATARIA É CRIME.

**Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.**

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



**Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.**